

PART 1

Software Side-Channel Attacks

1

Timing Attacks

Daniel PAGE

University of Bristol, United Kingdom

Set within the context of cryptographic engineering, timing attacks are a class of side-channel attack: if the execution latency of some target operation depends on security-critical data, this may allow an attacker to leverage passive execution latency measurements in any attempt to recover said data. Timing attacks have a rich history, and, in part because variation in execution latency can arise from a wide range of sources, remain surprisingly stubborn. Although a known problem, they remain far from a solved problem as evidenced by related instances continuing to appear more than 20 years since initial publications on the topic. The aim of this chapter is to offer an introduction to that topic, covering *both* a set of foundational concepts from a more theoretical perspective *and* a set of example attacks and their mitigation (via countermeasures) from a more applied perspective. As a result, the content should enable you to understand and apply the concepts covered in a broader context, for example, within your own work, and beyond the specific, limited set of examples used.

1.1. Foundations

In this section, we focus on explanation of the central concepts. The aim, in fairly non-technical terms, is to first explain *what* a timing attack is, and then establish a terminology and model for characterizing and reasoning about them. Doing so offers a foundation, which we will then build on in section 1.2 through an investigation of more concrete, example attacks.

Embedded Cryptography 1,

coordinated by Emmanuel PROUFF, Guénaél RENAULT,
Matthieu RIVAIN and Colin O'FLYNN. © ISTE Ltd 2025.

```

1 algorithm BUBBLE( $X, n$ ) begin
2   do
3      $f \leftarrow \text{false}$ 
4     for  $i = 1$  upto  $n - 1$  do
5       if  $X_{i-1} > X_i$  then
6          $X_{i-1} \leftrightarrow X_i$ 
7          $f \leftarrow \text{true}$ 
8       end
9     end
10    while  $f = \text{true}$ 
11 end

```

(a) A more abstract description of bubble sort, using pseudo-code

```

1 void bubble( int* X, int n ) {
2   bool f;
3
4   do {
5     f = false;
6
7     for( int i = 0; i < n; i++ ) {
8       if( X[ i - 1 ] > X[ i ] ) {
9         int t = X[ i - 1 ];
10        X[ i - 1 ] = X[ i ];
11        X[ i ] = t;
12      }
13      f = true;
14    }
15  }
16  while( f == true );
17 }

```

(b) A more concrete description of bubble sort, using C source code

Figure 1.1. Two descriptions of bubble sort

1.1.1. Execution latency in theory

DEFINITION 1.1.– Given a computational process P , the term *execution latency* is a measure of the delay between P starting and finishing execution (whether it completes normally or is terminated abnormally).

We intentionally leave the *unit* of measurement undefined. That is, within different contexts it might be appropriate to use different units, for example, an abstract unit such as computational steps, a concrete, human-oriented unit such as seconds, or a concrete, machine-oriented unit such as clock cycles. If the unit relates to time in a meaningful way, then it is common to see *execution time* used synonymously with *execution latency*: both terms capture the idea that we measure how much time elapses between starting and finishing execution.

The bubble sort algorithm is conceptually simple, so is an appealing, generic example to use as a vehicle to explore this concept further. The algorithm, described in Figure 1.1(a) using pseudo-code, repeatedly iterates over an n -element input array X , comparing adjacent elements and swapping them in-place if they occur in the wrong order; this is repeated until the array is sorted, which we know is true when no more swaps are required.

So, given an input array, what is the execution latency of this algorithm? That is, how much time elapses between starting (using X and n as input) and finishing execution (meaning a sorted X is produced as output) of the algorithm? Analysis of computational complexity (or, more specifically, time complexity) offers one

approach to providing an answer. Doing so gives a result in terms of abstract computational steps, focused on the problem size n . By using such analysis, we conclude that in the worst case *and* average case, the algorithm will perform $O(n^2)$ comparison operations and $O(n^2)$ swap operations; in the best case, the algorithm will perform $O(n)$ comparison operations and $O(1)$ swap operations.

DEFINITION 1.2.— The execution latency of a computational process P is termed *data dependent* if it depends on, and so may vary as a result of, the input data. Otherwise, it is termed *data independent*: this implies that the execution latency is the same, irrespective of the input data for every execution of P .

Two facts should be clear from our analysis of bubble sort. First, and more obviously, the *length* of X impacts the execution latency: in all cases, the number of steps will be greater for an X with many elements (i.e. larger n) than it will for an X with few elements (i.e. smaller n). Second, and less obviously, however, the *content* of X *also* impacts the execution latency: as suggested by the differing best and worse cases, the number of steps will be less for an X which is already (close to being) sorted than it will for an X , which is not. Either way, execution latency of the bubble sort algorithm would therefore be classified as data dependent.

1.1.2. Execution latency in practice

DEFINITION 1.3.— In concrete terms, data-dependent execution latency arises from (at least) the following cases: (1) *which operations are executed*, as determined by control-flow decisions made during execution; and (2) *how operations are executed*, or, put another way, what the execution latency of each executed operation is and whether or not it is data dependent.

Although instances of the first case can be identified within an abstract algorithm, instances of the second case are dependent on concrete details of an associated implementation and platform it is executed on. Conceptually at least, we could view this shift as including rather than (intentionally) excluding constant factors in analysis using computational complexity. Doing so is clearly imperative, because we ultimately aim to reason about concrete, real-world versus abstract, on-paper scenarios.

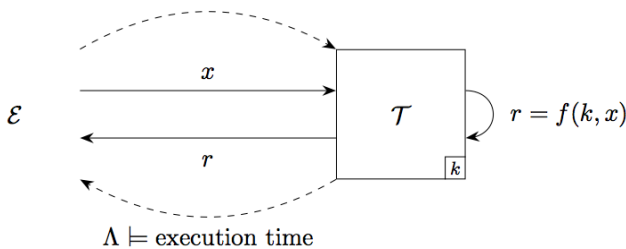
We can identify instances of both cases using Figure 1.1(b), which describes an implementation of bubble sort using C source code. For example, the `if` statement on line 8 represents an example of the first case: the flow of control is managed so lines 9–13 are either executed or not, depending on whether or not $X[i - 1] > X[i]$. Likewise, the memory access implied by the expressions $X[i - 1]$ and $X[i]$ on line 8 represents an example of the second case: the latencies of said accesses depend, for example, on the memory hierarchy (e.g. the

existence and state of any caches that exist within it). More generally, the platform will optimize instruction execution; it may do so for both memory access *and* computational instructions (for example, although this implementation does not include any, shift, multiplication, and division instructions sometimes exhibit data-dependant execution latency). An implication of this fact is that robust analysis of data (in)dependence cannot arise from the (static) source code alone. The Instruction Set Architecture (ISA) said source code ultimate targets will typically only capture functional properties: how instructions will be (dynamically) executed, and therefore the behavioral properties of the platform (i.e. a specific micro-architectural implementation of that ISA), *also* need to be assessed.

1.1.3. Attacks that exploit data-dependent execution latency

In a sense, bubble sort is as efficient as it can be: it only performs the steps required to produce a correct output from an associated input by exiting the outer-loop as soon as no more swaps are required. Adopting a perspective where efficiency is the metric of interest, such an approach is clearly positive. In fact, do *any* negatives stem from it? If we adopt a different perspective where security is the (or at least a) metric of interest, the answer is yes.

Put simply, the fact that execution latency is data dependent implies some form of correlation: measuring the execution latency will provide information, of some form, about the data it depends on. Set in some scenario where an adversary has access to execution latency measurements, this can be hugely problematic. Specifically, consider a scenario where the data involved are security critical, for example, but without loss of generality, some cryptographic key material. Using bubble sort as an example starts to become tenuous at this point, so instead consider the following generic attack model:



which helps fix various concepts and notation. $\mathcal{E}$ is using $\mathcal{T}$ to perform some computation f . Having first accepted an input x from $\mathcal{E}$, $\mathcal{T}$ computes and then provides $r = f(k, x)$ as output. $\mathcal{E}$ can also measure the execution latency associated with computation by $\mathcal{T}$, which we denote Λ . On the one hand, k is not and *should* not be known by $\mathcal{E}$. On the other hand, however, Λ may provide information about k if the execution latency of f is data dependent, and therefore may potentially be

exploited in an attack aiming to recover it. This is the crux of a side-channel attack based on execution latency, or, less formally, a *timing attack*. We say that information about the security-critical data is leaked (or is leakage) through a side-channel represented by execution latency. Such attacks are potent, because, as a result of exploiting the additional information afforded by the side-channel, they can often be significantly more efficient than alternatives (e.g. traditional cryptanalysis).

Now focused on the topic of side-channel attacks, we need to address two important points regarding terminology. First, in relation to data (in)dependence, the term data are qualified by the fact that we almost always mean security-critical data. There is limited value in learning information about data we already know, for example, and so no implication for execution latency depending on it. So, from here on, read “data” as “security-critical data” unless otherwise stated. Second, it is very common to see the term constant-time (respectively, variable-time) used as a synonym for data-independent (respectively, data-dependent) execution latency in this context (already ignoring the clash vs. $O(1)$ in computational complexity). We avoid these terms, deeming data (in)dependence more reflective of what is actually intended. For example, the execution latency of a given algorithm *may* vary and need not reflect any *specific* constant: we only demand that it does *not* vary, and so is (some) constant with respect to security-critical data involved. Likewise, use of the term constant-time suggests execution time (or latency) is the *only* pertinent issue; in reality, and more generally, we need to address the challenge of data-dependent resource use (*both* in terms of which resources are used, *and* the length of time they are used for). We can find examples of this fact investigated in Chapter 2.

Having presented the high-level concept, we now refine various lower level details of our attack model. We do so in a piecemeal manner below, framing the discussion around a set of concise definitions which, in combination, help to explain and characterize specific attacks (such as those used as examples in section 1.2).

DEFINITION 1.4.— The attack strategy employed by $\mathcal{E}$ may be described as *differential* or *non-differential*.

At a high level, a non-differential attack will typically use few (even just one) execution latency measurements in isolation. In contrast, a differential attack will typically use many execution latency measurements in combination, for example, through analysis of when and how they differ as the result of data-dependent behavior by $\mathcal{T}$.

DEFINITION 1.5.— The attack strategy employed by $\mathcal{E}$ may be assessed, where relevant, in relation to either *efficacy* and/or *efficiency*.

Although exceptions might exist, efficacy is normally a binary assessment: either the attack by $\mathcal{E}$ is successful with respect to some stated aim or not. In contrast, there

are many ways to assess efficient resource use by $\mathcal{E}$ during said attack. Focusing on time, for example, we might view the number of interactions with $\mathcal{T}$, the high-level, algorithm-related attack latency (e.g. stated in terms of computational complexity) or the low-level, implementation-related attack latency (e.g. stated in terms of wall-clock time), as indicative of how efficient $\mathcal{E}$ is.

DEFINITION 1.6.– If the interaction between $\mathcal{E}$ and $\mathcal{T}$ is direct, the attack is described as *local*; otherwise, if the interaction is indirect, the attack is described as *remote*.

There are other, reasonable ways to capture a similar concept, for example, using physical proximity (implying physical access) as the measure of localness. We opt for the above instead, because $\mathcal{E}$ and $\mathcal{T}$ may be in close physical proximity but *still* physically or logically isolated from each other; this would imply an indirect form of interaction. Either way, two points are important. First, although the attack model above at least *suggests* that $\mathcal{E}$ and $\mathcal{T}$ interact remotely, this is *not* a limitation. For example, $\mathcal{E}$ may leverage physical access to some device $\mathcal{T}$ in order to mount a local attack; $\mathcal{E}$ and $\mathcal{T}$ may be software processes (co-)residing and so executing locally on the same platform; $\mathcal{E}$ and $\mathcal{T}$ may be software processes residing and so executing on different platforms, interacting remotely via an intermediate network. Second, if/when a remote attack is feasible, this is often viewed as an advantage. For example, such an attack typically removes the need for physical access to $\mathcal{T}$, avoids any reliance on additional equipment (for example, see power- or EM-based side-channels) and allows attacks to be applied at greater scale (in the sense there are likely to be more accessible instances of $\mathcal{T}$ to attack).

DEFINITION 1.7.– From the perspective of $\mathcal{E}$, computation by $\mathcal{T}$ may be described as *synchronous* or *asynchronous*.

The synchronous case suggests that $\mathcal{E}$ knows or even *controls* when $\mathcal{T}$ starts or finishes execution; in contrast, the asynchronous case suggests that $\mathcal{T}$ operates independently from $\mathcal{E}$ (and, as a result, is often described as free-running). In the asynchronous case, one typically attempts to identify some form of trigger or reference event that indicates the start and/or finish of execution. Such an event might be observable via a secondary side-channel, or simply observation of when and how $\mathcal{T}$ interacts with external parties (such as $\mathcal{E}$) or resources (such as memory or peripheral devices).

DEFINITION 1.8.– The execution latency measurements taken by $\mathcal{E}$ are characterized by their *accuracy*, that is, how close the measured and actual execution latencies are, and *precision*, that is, how close two execution latency measurements for the same computation are. The former case typically relates to systematic error, for example, due to the impact of *quantization*; whereas the latter case typically relates to random error, for example, due to the impact of *noise*.

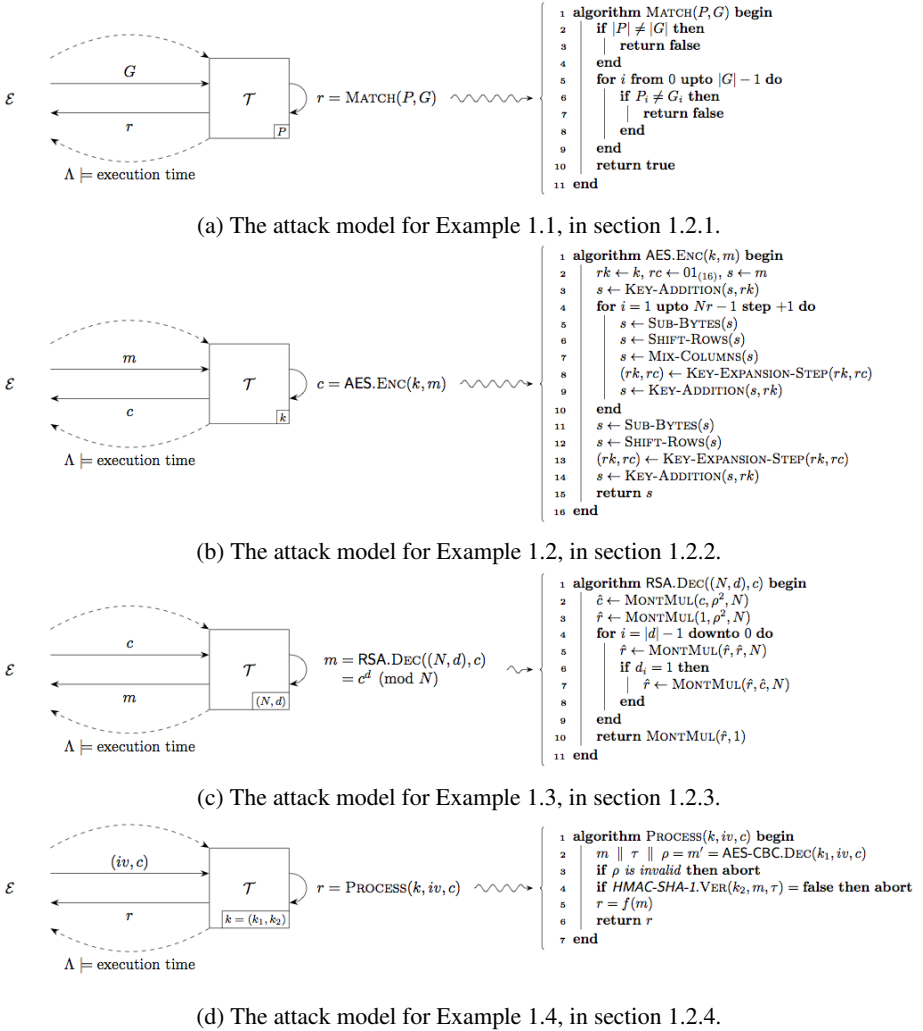


Figure 1.2. Attack models relating to the examples presented in section 1.2

We deem any formal exploration of what noise *is* beyond our scope; it suffices to understand that noise will degrade accuracy and precision, in the sense that execution latency of the same computation within different interactions between $\mathcal{E}$ and $\mathcal{T}$ may differ (or fluctuate). The term Signal-to-Noise Ratio (SNR) is often used to capture the relative strength of the signal of interest, that is, the actual execution latency, versus any noise that causes the measured execution latency to differ. Both statistical and non-statistical techniques can typically be harnessed in an attempt to reduce or even

remove noise, that is, yield higher SNR; examples include averaging and filtering. Doing so can be important, because, ultimately, SNR will typically have some impact on the efficiency and/or efficacy of most attacks.

1.2. Example attacks

In this section, we focus on application of the central concepts: the aim is to illustrate *how* timing attacks are mounted. Harnessing the foundation offered by section 1.1, we do so by considering a small set of examples each framed within a simple but plausible scenario; we structure each example in the same way by (1) defining the attack model, (2) analyzing the attack model, for example, to characterize the source of data dependence and then, finally, (3) presenting an attack based on said analysis.

1.2.1. Example 1.1: an explanatory attack on password validation

1.2.1.1. Scenario

Consider the attack model depicted in Figure 1.2(a): an adversary $\mathcal{E}$ interacts with a target device $\mathcal{T}$ that stores some embedded, security-critical data P (a password). When provided with input G (a guess at the password), $\mathcal{T}$ first performs the computation detailed by the algorithm MATCH (that is, string matching, or, more specifically password validation) and then responds with r as output. $\mathcal{E}$ can *also* measure the execution latency of computation performed by $\mathcal{T}$, so attempts to exploit this information with the aim of recovering P (implying, for example, it then gains access to $\mathcal{T}$).

1.2.1.2. Analysis

In the same way we reasoned about bubble sort, the MATCH algorithm is as efficient as it can be: as soon as it detects a mismatch between P and G , it immediately returns *false* rather than needlessly continuing. Such a mismatch can occur for two reasons. First, and resulting in execution of line 3, the mismatch might stem from the *length* of, that is, number of characters in, P and G : if $|P| \neq |G|$, then the two cannot match. Second, and resulting in execution of line 7, the mismatch might arise from the *content* of P and G : if $P_i \neq G_i$ for some i , then the two cannot match. This is termed early-abort behavior, and is possible when a special-case condition means the general-case algorithm need not be fully executed in order to provide correct output from the associated input. Viewed from the perspective of efficiency, this behavior is positive; when viewed from the perspective of security, the same behavior becomes negative; however, it means that the execution latency of MATCH is data dependent, that is, depends on P and G .

1.2.1.3. Exploitation

Assume, without loss of generality, that the set of valid characters is $A = \{'a', 'b', \dots, 'z'\}$, and, specifically, that $P = \text{"pencil"}$, which means $|P| = 6$. The attack proceeds in two phases:

1) In this phase, the attack aims to recover $|P|$. To do so, we use a sequence of guesses:

$$G \in \langle "a", "aa", "aaa", "aaaa", \dots \rangle$$

during interaction with $\mathcal{T}$, that is, the i th guess is $i + 1$ characters in length. The execution latency for guess $G = \text{"aaaaaa"}$ will be longer than for the others because $|P| = |G|$; this means execution for it will progress into the loop, rather than early-abort at line 3. As such, identifying this G recovers $|P|$.

2) In this phase, the attack aims to recover P iteratively, that is, character-by-character. To recover P_i in iteration $i = 0$, we use a sequence of guesses:

$$G \in \langle \text{"aaaaaaa"}, \text{"baaaaaa"}, \text{"caaaaaa"}, \dots, \text{"zaaaaaa"} \rangle$$

during interaction with $\mathcal{T}$, that is, the i th character cycles through all possibilities, while the others remain fixed. The execution latency for guess $G = \text{"paaaaaa"}$ will be longer than for the others, because $|P| = |G|$ and $P_0 = G_0$; this means execution will progress into loop iteration $i + 1 = 1$, rather than early-abort at line 7 in iteration $i = 0$. As such, $G_0 = \text{'p'}$ is deemed correct in this guess, which we now fix. To recover P_i in iteration $i = 1$, we use a sequence of guesses:

$$G \in \langle \text{"paaaaaa"}, \text{"pbaaaaa"}, \text{"pcaaaaa"}, \dots, \text{"pzaaaaa"} \rangle$$

during interaction with $\mathcal{T}$, that is, the i th character cycles through all possibilities, while the others remain fixed. The execution latency for guess $G = \text{"peaaaaa"}$ will be longer than for the others, because $|P| = |G|$, $P_0 = G_0$ and $P_1 = G_1$; this means execution will progress into loop iteration $i + 1 = 2$, rather than early-abort at line 7 in iteration $i = 1$. As such, $G_1 = \text{'e'}$ is deemed correct in this guess, which we now fix. The attack continues in the same way for $0 \leq i < n$, until P is eventually fully recovered.

Since this is an explanatory attack, it is easy to assess it in terms of both efficacy and efficiency versus intuitive alternatives. For example, we could consider a brute-force attack: by essentially trying every n -character guess, this strategy assumes knowledge of n , requires $O(|A|^n)$ guesses and guarantees recovery of P . Or, we could consider a dictionary attack: by first constructing a dictionary of common (or likely) passwords $D = \{\text{"password"}, \text{"qwerty"}, \text{"admin"}, \dots\}$ to use as guesses, this strategy does not assume knowledge of n , requires $O(|D|)$ guesses and does not guarantee recovery of P . In contrast, our side-channel attack does not assume knowledge of n , requires $O(|A| \cdot n)$ guesses and guarantees recovery of P ; in a sense, the additional information allows it to offer greater efficiency than the brute-force attack *and* greater efficacy than the dictionary attack.

1.2.2. Example 1.2: an attack on `xtime`-based AES

1.2.2.1. Scenario

Assuming use of AES-128 throughout, consider the attack model depicted in Figure 1.2(b): an adversary $\mathcal{E}$ interacts with a target device $\mathcal{T}$, which stores some embedded, security-critical data k (an AES cipher key). When provided with input m (an AES plaintext), $\mathcal{T}$ first performs the computation detailed by the algorithm `AES.ENC` (i.e. AES encryption) and then responds with c (an AES ciphertext) as output. $\mathcal{E}$ can *also* measure the execution latency of computation performed by $\mathcal{T}$, so we attempted to exploit this information with the aim of recovering k .

1.2.2.2. Analysis

AES is an iterative block cipher based on an SP network; as one of three possible parameterizations, AES-128 uses a 16 byte cipher key and $b = 16$ byte block size. At a low level, it operates on elements in the finite field $\mathbb{F}_{2^8}$ realized as $\mathbb{F}_2[\mathbf{x}]/p(\mathbf{x})$, where $p(\mathbf{x}) = \mathbf{x}^8 + \mathbf{x}^4 + \mathbf{x}^3 + \mathbf{x} + 1$. A short-hand is often adopted for elements of this field, meaning, for example, $a(\mathbf{x}) = \mathbf{x}^4 + \mathbf{x} + 1 \equiv 13_{(16)}$, suggesting that each element can be represented by a byte (whose i th bit represents the i th coefficient). Elements of $\mathbb{F}_{2^8}$ are collected into (4×4) -element state and round key matrices; the i th row and j th column of such a matrix relating to round r is denoted $s_{i,j}^{(r)}$ and $rk_{i,j}^{(r)}$ respectively, with super- and/or subscripts omitted whenever irrelevant. The process of encryption initializes $s^{(0)} = m$ and $rk^{(0)} = k$, in both cases filling the left-hand matrix in a column-wise manner using content from the right-hand sequence of bytes.

$\mathcal{T}$ adopts an implementation strategy common in constrained platforms, targeting a balance that trades-off higher execution latency for lower memory footprint. This can be summarized as follows. First, it pre-computes the S-box: doing so implies it is realized using a table look-ups, rather than any computation. Second, it updates (or evolves) the cipher key, computing each round key during encryption rather than pre-computing them. Third, and finally, it uses the algorithm:

```

1 algorithm XTIME(a) begin
2   if  $a_7 = 1$  then
3     return  $(a \ll 1) \oplus 11B_{(16)}$ 
4   else
5     return  $(a \ll 1)$ 
6   end
7 end
```

to compute the multiplication-by- $\mathbf{x}$ operation, that is, $a(\mathbf{x}) \cdot \mathbf{x} \pmod{p(\mathbf{x})}$. On lines 3 and 5, the unconditional left-shift captures multiplication by $\mathbf{x}$. On line 3, however,

the conditional XOR captures a reduction modulo $p(\mathbf{x})$; note that such a reduction is only required when $a_7 = 1$. Using this algorithm, it applies:

$$\begin{aligned} \text{MIX-COLUMN} \left(\begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix} \right) &= \begin{bmatrix} 02_{(16)} & 03_{(16)} & 01_{(16)} & 01_{(16)} \\ 01_{(16)} & 02_{(16)} & 03_{(16)} & 01_{(16)} \\ 01_{(16)} & 01_{(16)} & 02_{(16)} & 03_{(16)} \\ 03_{(16)} & 01_{(16)} & 01_{(16)} & 02_{(16)} \end{bmatrix} \times \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix} \\ &= \begin{bmatrix} \text{xtime}(s_{0,j}) \oplus (\text{xtime}(s_{1,j}) \oplus s_{1,j}) \oplus s_{2,j} \oplus s_{3,j} \\ s_{0,j} \oplus \text{xtime}(s_{1,j}) \oplus (\text{xtime}(s_{2,j}) \oplus s_{2,j}) \oplus s_{3,j} \\ s_{0,j} \oplus s_{1,j} \oplus \text{xtime}(s_{2,j}) \oplus (\text{xtime}(s_{3,j}) \oplus s_{3,j}) \\ (\text{xtime}(s_{0,j}) \oplus s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus \text{xtime}(s_{3,j}) \end{bmatrix} \end{aligned}$$

to each j th column of the state within MIXCOLUMNS.

In summary, the execution latency of AES encryption is data dependent: this stems from the implementation strategy adopted, whereby the execution latency of `xtime` and therefore MIX-COLUMNS is data dependent.

1.2.2.3. Exploitation

The attack strategy aims to recover k iteratively, that is, byte-by-byte. Let k_j and m_j denote the j th byte of plaintext and cipher key, respectively. In any t th iteration of the attack, we aim to recover the target byte k_t based on the observation that, in the first MIX-COLUMNS invocation, the computation:

$$\text{xtime}(\text{S-BOX}(m_t \oplus k_t))$$

is performed at least once; we term this the target `xtime`. Doing so involves the following steps, parameterized by N and M :

- 1) Compute the $(256 \times N)$ -element matrix $\mathcal{H}$ such that:

$$\mathcal{H}_{i,j} = \begin{cases} 1 & \text{if } \text{S-BOX}(i \oplus j)_7 = 1 \\ 0 & \text{if } \text{S-BOX}(i \oplus j)_7 = 0 \end{cases}$$

Across the rows, each $0 \leq i < 256$ captures a possible value of k_t ; across the columns, each $0 \leq j < N$ captures a possible value of m_t (meaning $N = 256$ will consider *all* possible values). As such, each $\mathcal{H}_{i,j}$ captures whether or not the target `xtime` performs a conditional XOR for specific k_t and m_t .

- 2) Compute the $(N \times M)$ -element matrix:

$$\mathcal{S}_{i,j} = \{ \langle s_0, s_1, \dots, s_{b-1} \rangle \mid s_k = i \text{ if } k = t, s_k \text{ is random if } k \neq t \}$$

of plaintexts. So, since all plaintexts in the i th row have the same t th byte, they all yield the same input to the target `xtime`. Let $\mathcal{R}_{i,j}$ denote the execution latency arising from encryption of $\mathcal{S}_{i,j}$, which we measure via interaction with $\mathcal{T}$, and $\bar{\mathcal{R}}_i$ denote the mean of execution latencies across the i th row of $\mathcal{R}$.

The idea is that, provided N and M are large enough, $\bar{\mathcal{R}}_i$ will reflect the target `xtime`, that is, it will be larger or smaller depending on whether or not the conditional XOR occurs. Put another way, with some probability of error, $\bar{\mathcal{R}}_i$ indicates the value of $\text{S-BOX}(i \oplus k_t)_7$. To recover k_t , we compare each $\bar{\mathcal{R}}$ with each $\mathcal{H}_i$, for example, using the Pearson correlation coefficient: the i which yields the strongest correlation indicates the value of k_t .

1.2.3. Example 1.3: an attack on Montgomery-based RSA

1.2.3.1. Scenario

Consider the attack model depicted in Figure 1.2(c): an adversary $\mathcal{E}$ interacts with a target device $\mathcal{T}$, which stores the embedded, security-critical data (N, d) (an RSA private key). When provided with input c (an RSA ciphertext), $\mathcal{T}$ first performs the computation detailed by the algorithm `RSA.DEC` (i.e. RSA decryption) and then responds with m (an RSA plaintext) as output. $\mathcal{E}$ can *also* measure the execution latency of computation performed by $\mathcal{T}$, so attempts to exploit this information with the aim of recovering d .

1.2.3.2. Analysis

$\mathcal{T}$ uses the left-to-right, binary (or square-and-multiply) exponentiation algorithm. Note that all modular squaring and multiplication operations, for example, in lines 4 and 6, are based on the use of Montgomery reduction. This requires pre-computation of ω and ρ from N , such that $\hat{x} = x \cdot \rho \pmod{N}$ then denotes the Montgomery representations of some x .

The execution latency of modular exponentiation is data dependent, because the execution of line 6 is conditional on d_i . An execution latency measurement may therefore allow us to infer how often line 6 is executed, and hence the Hamming weight of (or number of bits equal to 1 in) d . Beyond this, we need to analyze the `MONTMUL` algorithm as used to support the exponentiation algorithm. The steps involved are captured as follows:

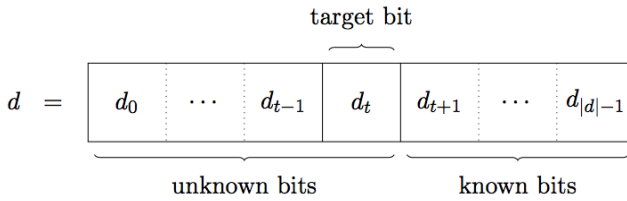
```

1 algorithm MONTMUL( $x, y, N$ ) begin
2    $r \leftarrow x \cdot y$ 
3    $r \leftarrow (r + ((r \cdot \omega) \bmod \rho) \cdot N) / \rho$ 
4   if  $r \geq N$  then  $r \leftarrow r - N$ 
5   return  $r$ 
6 end
```

where line 2 performs an integer multiplication, line 3 performs a Montgomery reduction, and then, finally, line 4 performs a conditional subtraction to fully reduce r modulo N . From this, we conclude that the execution latency of MONTMUL is also data dependent, because the execution of line 4 is dependent on both the operands x and y and the modulus N ; as used to support exponentiation, this implies dependence on c (because it forms one of the operands), and, crucially, d (because it controls when, where and how MONTMUL is invoked).

1.2.3.3. Exploitation

The attack strategy aims to recover d iteratively, that is, bit-by-bit, working from the most-significant toward the least-significant bit. In any t th iteration of the attack, d is partially known and partially unknown, which we can depict as follows:



That is, we already know some more-significant bits of d (noting that it must be the case that some $d_{|d|-1} = 1$; otherwise $d = 0$) and so aim to recover the next less significant, as yet unknown target bit d_t . Let $c[k]$ denote some k th ciphertext, randomly selected modulo N for $0 \leq k < n$; for each such $c[k]$, we use (partial) knowledge of d to simulate a (partial) decryption as would be performed by $\mathcal{T}$. Doing so involves the following steps:

1) Start by setting $c = c[k]$, then simulating lines 2 and 3, that is, computing $\hat{c} \leftarrow \text{MONTMUL}(c, \rho^2, N)$ and $\hat{r} \leftarrow \text{MONTMUL}(1, \rho^2, N)$.

2) Step the simulation forward until we reach the start of loop iteration t (between lines 4 and 5). We can do so because d_i is known for $t < i < |d|$, meaning the simulation can perform exactly the same computation that $\mathcal{T}$ would.

3) Step the simulation forward until we reach the end of loop iteration t (between lines 8 and 9). We can do so because although d_i is unknown for $i = t$, we can fork the simulation to reflect guessing both possibilities:

a) for the simulation fork that guesses $d_i = 0$, line 6 is not executed so we do not update $\hat{r}$;

b) for the simulation fork that guesses $d_i = 1$, line 6 is executed so we update $\hat{r} \leftarrow \text{MONTMUL}(\hat{r}, \hat{c}, N)$.

4) Step the simulation forward until we reach the middle of loop iteration $t + 1$ (between lines 5 and 6). That is, we update $\hat{r} \leftarrow \text{MONTMUL}(\hat{r}, \hat{r}, N)$ and, in doing

so, record whether or not the conditional subtraction, that is, line 4 of MONTMUL, is executed.

5) Classify c by placing it into set:

$\mathcal{S}_0$ if the simulation fork guessed $d_i = 0$ and no conditional subtraction is executed
 $\mathcal{S}_1$ if the simulation fork guessed $d_i = 0$ and conditional subtraction is executed
 $\mathcal{S}_2$ if the simulation fork guessed $d_i = 1$ and no conditional subtraction is executed
 $\mathcal{S}_3$ if the simulation fork guessed $d_i = 1$ and conditional subtraction is executed

This means c is placed into exactly *two* sets: either $\mathcal{S}_0$ or $\mathcal{S}_1$ arising from the simulation fork that guessed $d_i = 0$, and either $\mathcal{S}_2$ or $\mathcal{S}_3$ arising from the simulation fork that guessed $d_i = 1$.

The idea is that we have intentionally constructed the sets so there should be a difference between the execution latency for the ciphertexts in $\mathcal{S}_0$ versus those in $\mathcal{S}_1$, and the ciphertexts in $\mathcal{S}_2$ versus those in $\mathcal{S}_3$. Under the assumption that conditional subtractions will occur more or less at random during exponentiation, ciphertexts in the latter will, on average, execute an extra conditional subtraction versus those in the former. However, only one of the guesses made by the simulation is actually correct, so that difference should only be evident in either the first or second case. Put another way, for the correct (respectively, incorrect) guess, the simulated computation will (respectively, will not) match the actual computation by $\mathcal{T}$; therefore, for the correct (respectively, incorrect) guess, the number of conditional subtractions that occur during exponentiation of ciphertexts in one set will differ from (respectively, be approximately equal to) those in the other set. So, let

$$\Lambda(\mathcal{S}_k) = \frac{1}{|\mathcal{S}_k|} \sum_{c \in \mathcal{S}_k} \Lambda(c)$$

denote the average execution latency of ciphertexts in set $\mathcal{S}_k$, where $\Lambda(c)$ denotes the execution latency for some ciphertext c : we measure this by interacting with $\mathcal{T}$. One of three cases will apply:

$$\begin{aligned} \text{abs}(\Lambda(\mathcal{S}_0) - \Lambda(\mathcal{S}_1)) &> \text{abs}(\Lambda(\mathcal{S}_2) - \Lambda(\mathcal{S}_3)) &\Rightarrow d_t = 0 \\ \text{abs}(\Lambda(\mathcal{S}_0) - \Lambda(\mathcal{S}_1)) &< \text{abs}(\Lambda(\mathcal{S}_2) - \Lambda(\mathcal{S}_3)) &\Rightarrow d_t = 1 \\ \text{abs}(\Lambda(\mathcal{S}_0) - \Lambda(\mathcal{S}_1)) &\simeq \text{abs}(\Lambda(\mathcal{S}_2) - \Lambda(\mathcal{S}_3)) &\Rightarrow d_t = ? \end{aligned}$$

In the first (top) and second (middle) cases, the difference we expect will allow us to infer the value of d_t . In the third (bottom) case, however, the difference is too close to allow confident inference of d_t ; this may be due to, for example, use of too small an n or the impact of noise.

Having recovered d_t , we proceed with the next, $(t + 1)$ th attack iteration and recover the next, $(t + 1)$ th bit of d in the same way; this continues until d is eventually fully recovered.

1.2.4. Example 1.4: a padding oracle attack on AES-CBC

1.2.4.1. Scenario

Assuming use of AES-128 throughout, consider the attack model depicted in Figure 1.2(d): an adversary $\mathcal{E}$ interacts with a target device $\mathcal{T}$, which stores the embedded, security-critical data k (some key material). When provided with input (iv, c) , $\mathcal{T}$ first performs the computation detailed by the algorithm PROCESS, then responds with r as output. $\mathcal{E}$ obtains a ciphertext (iv^*, c^*) , which is the encryption of some unknown plaintext m^* under k . It can *also* measure the execution latency of computation performed by $\mathcal{T}$, so attempts to exploit this information with the aim of recovering m^* .

1.2.4.2. Analysis

Let $x[i]_j$ denote the j th byte within the i th block of some n -block plaintext or ciphertext x ; this means $0 \leq i < n$ and $0 \leq j < b$, assuming a b -byte block size.

At a high level, PROCESS can be described line-by-line as follows. Line 2 decrypts the ciphertext c , parsing the result m' as a plaintext m , some padding ρ and a tag τ . Then, line 3 checks whether ρ is valid, aborting if not; line 4 checks whether τ is valid, aborting if not; and, finally, line 5 processes m using some function f to produce r . At a low level, some further details are important: note that line 2 decrypts c using AES in Cipher Block Chaining (CBC) mode using the key k_1 , and line 3 assumes τ is a SHA1-based HMAC Message Authentication Code (MAC) tag and checks validity accordingly using the key k_2 . The padding scheme used to specify ρ is also important, but not detailed within the description of PROCESS. Although alternatives exist, assume a TLS-like scheme is used: let

$$\rho(\alpha) = \underbrace{\langle \alpha, \alpha, \dots, \alpha \rangle}_{\alpha \text{ byte(s)}}$$

denote a valid padding sequence, which contains α byte(s) equal to α and then 1 compulsory byte equal to α , which records the total padding length. Note that if $m \parallel \tau$ is l bytes long, then $\rho = \rho(b - (l + 1) \bmod b)$.

The execution latency of PROCESS is data dependent: if t_3 denotes the execution latency when $\mathcal{T}$ early-aborts on line 3 based on ρ , t_4 denotes the execution latency when $\mathcal{T}$ early-aborts on line 4 based on τ , and t_5 denotes the execution latency when $\mathcal{T}$ executes line 5, then, irrespective of their concrete values, $t_3 < t_4 < t_5$. In fact,

we can focus on the ability to distinguish between these cases by defining a so-called padding oracle: for the scenario at hand, this would be something like:

$$\mathcal{O}(iv, c) = \begin{cases} \text{true} & \text{if AES-CBC.DEC}(k_1, iv, c) \text{ has valid padding} \\ \text{false} & \text{if AES-CBC.DEC}(k_1, iv, c) \text{ has invalid padding} \end{cases}$$

Doing so can be viewed as an abstraction of the underlying mechanism used to measure execution latency, thus separating it from the attack strategy.

1.2.4.3. Exploitation

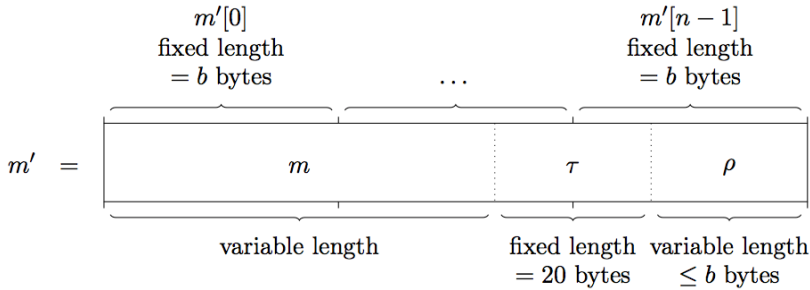
Use of AES in CBC mode means that $\mathcal{T}$ will compute:

$$m'[i] = \text{AES.DEC}(k_1, c[i]) \oplus x_i$$

where

$$x_i = \begin{cases} iv & \text{if } i = 0 \\ c[i - 1] & \text{if } i > 0 \end{cases}$$

for $0 \leq i < n$. The resulting plaintext m' is then parsed into the fields m , τ , and ρ , which we can depict as follows:



From this specific example, we can infer several more general facts. First, ρ will occur in $m'[n-1]$, that is, the last, $(n-1)$ th block of m' . Second, although $|\rho|$ is unknown, we *do* know $|\rho| \leq b$ because the point of including it at all is to ensure $|m'| = 0 \pmod{b}$. Third, the definition of CBC mode means that:

$$m'[n-1] = \text{AES.DEC}(k_1, c[n-1]) \oplus x_i.$$

As such, by controlling x_i (meaning either iv or $c[n-2]$, depending on i), we can control $m'[n-1]$ and hence also the ρ then checked for validity by $\mathcal{T}$. For example,

consider a two-block ciphertext c . If we construct an input (iv, c') where $c' = (c[0] \oplus \delta) \parallel c[1]$, for some δ , that is, $c'[0] = c[0] \oplus \delta$ and $c'[1] = c[1]$, then $\mathcal{T}$ will compute:

$$\begin{aligned} m'[0] &= \text{AES.DEC}(k_1, c'[0]) \oplus iv \\ &= \text{AES.DEC}(k_1, c[0] \oplus \delta) \oplus iv \\ m'[1] &= \text{AES.DEC}(k_1, c'[1]) \oplus c'[0] \oplus \delta \\ &= \text{AES.DEC}(k_1, c[1]) \oplus c[0] \oplus \delta \end{aligned}$$

That is, $\mathcal{T}$ will compute an m' where $m'[0]$ is randomized by and $m'[1]$ is controlled by the δ selected; since $m'[1]$ is the $(n-1)$ th block in m' , δ will also control ρ and hence the padding oracle result, that is,

$$\mathcal{O}(iv, (c[0] \oplus \delta) \parallel c[1]).$$

Based on these facts, and assuming again that $|c^*| = 2$, the attack proceeds in two phases:

1) In this phase, the attack strategy constructs a so-called “recover last byte(s)” oracle. First, select a random b -byte block δ and search for a value of δ_{b-1} such that:

$$\mathcal{O}(iv, (c^*[0] \oplus \delta) \parallel c^*[1]) = \text{true}.$$

Since the padding of m' is deemed to be valid, we know that one of:

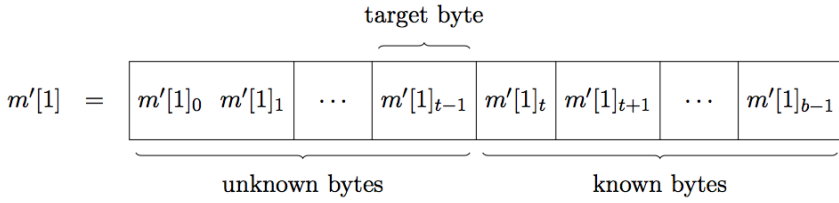
$$\begin{aligned} m'[1]_{b-1} &= m^*[1]_{b-1} \oplus \delta_{b-1} = \rho(0) \\ m'[1]_{b-2, \dots, b-1} &= m^*[1]_{b-2, \dots, b-1} \oplus \delta_{b-2, \dots, b-1} = \rho(1) \\ m'[1]_{b-3, \dots, b-1} &= m^*[1]_{b-3, \dots, b-1} \oplus \delta_{b-3, \dots, b-1} = \rho(2) \\ &\vdots \end{aligned}$$

is true: however, we do not know *which* one. So, to recover one (or more) byte(s) of m' , we need $|\rho|$, that is, the padding length. To recover it, start from $t = 0$ and alter (e.g. increment) δ_t . If

$$\mathcal{O}(iv, (c^*[0] \oplus \delta) \parallel c^*[1]) = \text{false},$$

then t marks the initial padding byte (because we just altered it, thus invalidating the padding) and implies that $|\rho| = b - t$; otherwise, increment t and try again.

2) In this phase, the attack strategy constructs a so-called “block decryption” oracle. As mentioned above, we have already recovered the most-significant $b - t$ bytes of m' , that is, the *natural* padding: we therefore have:



for some t where

$$m'[1]_{t,\dots,b-1} = m^*[1]_{t,\dots,b-1} \oplus \delta_{t,\dots,b-1} = \rho(b-t-1).$$

To recover the next least-significant byte of $m'[1]$, we try to control δ so an *extended* padding is produced. To do so, we begin by incrementing the known padding bytes by setting $\delta_j = m'[1]_j \oplus (b-t)$ for $t \leq j < b$, and then we apply a similar strategy as above by searching for a value of δ_{t-1} such that the padding of m' is deemed valid: this recovers $m'[1]_{t-1}$. Then, we can iterate for each next least-significant byte to recover the whole of $m'[1]$ (and hence $m^*[1]$).

Of course, as described, this process is specific to recovery of $m'[1]$ and, even then, the case where $|c^*| = 2$. However, with minor alterations the same underlying concept generalizes to cater for recovery of any $m'[i]$ and for any $|c^*|$.

1.3. Example mitigations

Fundamentally, mitigating side-channel attacks based on execution latency can be achieved by preventing said execution latency being data dependent. Although true, this vastly understates the challenge in doing so, which is evidenced by a variety of points. For example, first, the basic properties of scale and complexity can be hugely challenging. On the one hand, there is a large design space of techniques to consider; on the other hand, they often need to be applied to and so integrated with a large, complex software code base that may be constrained, for example, with respect to requirements of some overarching protocol. Second, the underlying issue is evident in, and so potentially must be addressed at various levels of abstraction, for example, both at a high level, relating to the design algorithms, *and* at a low level, relating to their implementation, and, ultimately execution. Third, security may be *a* metric, not *the* metric of interest; there can be pressure to compromise on security in order to meet efficiency targets, for example, which demands carefully considered trade-offs.

Although, there is seldom a panacea for such challenges, (1) clear understanding of the context, constraints, problem and platform, and (2) well-informed selection and application of implementation techniques are centrally important in producing an outcome, which is fit for purpose. We can find a detailed treatment of relevant topics in Chapters 7 of Volume 2. For completeness, however, the following provides

an illustrative overview of pertinent examples, framed around phases of a typical development cycle (implying iteration over said phases may be required).

1) *Design*: at a high(er) more abstract level, the first phase specifies a design for what will be implemented. Appropriate high-level choices will often have the largest impact from an efficiency perspective, and the same is true from a security perspective: care in this phase can make subsequent phases significantly easier.

Among many options, we might consider algorithm selection (in the context of ECC, e.g. use of the Montgomery ladder and/or complete addition formula), parameter selection (in the context of RSA, e.g. of ρ to remove conditional subtraction in Montgomery multiplication), techniques related to alternative representation of data (e.g. bit-slicing), techniques related to alternative computational models (e.g. based on URISC), approaches based on blinding (e.g. of an RSA private exponent), or approaches based on padding (e.g. to worst-case execution latency).

2) *Implementation*: at a low(er), more concrete level, the next phase implements what has been specified. Care with respect to how a high(er) level implementation is translated into a low-level instruction sequence by a tool chain, *and* how that instruction sequence is executed by the platform, is crucial. For example, note that many compiler optimizations intended to have a positive impact on efficiency *may* have a negative impact on security, e.g. by “un-doing” features intended to ensure data-independent execution latency.

Among many options, we might consider approaches based on control-flow linearization (or flattening) to yield straight-line code (e.g. software multiplexer gadgets, or via conditional-move instructions), approaches based on control-flow balancing, approaches that yield constant memory access patterns (e.g. within window-based modular exponentiation) and approaches that favor computation versus memory access (e.g. use of AES-NI versus look-ups into pre-computed S-box or T-table content).

3) *Evaluation*: in the final phase, we aim to evaluate what has been implemented, and, ideally, gather evidence and hence confidence that the associated execution latency is data independent.

Among many options, we might consider approaches based on (static) verification (including use of baseline assumptions, e.g. as provided by RISC-V Zkt pseudo-extension) and/or on (dynamic) experimentation (e.g. ctgrind and dudedec).

1.4. Notes and further references

– Section 1.1. There are many good side-channel attack overviews. Anderson (2020) offers a accessible, general example with a range of historical context, while Koeune (2011) covers attacks based on execution latency specifically. The first

publication related to such attacks was by Kocher (1996). Nearly a decade later, in Brumley and Boneh (2003) then expanded the scope by demonstrating remote applicability, that is, over a network. The same general concept has been further explored within numerous different scenarios, for example, by Schindler (2000), Aciicmez et al. (2005) and Brumley and Tuveri (2011). The work of Kocher is seminal, which presents both specific attacks, for example, on RSA and DSA, and articulates the more general threat; we can make a strong argument that it represents the foundation of modern research on side-channel attacks. For example, it already notes that “*RAM cache hits can produce timing characteristics in implementations*” and thus identifies the more general challenge of data-dependent execution latency arising from the platform; the range of components studied along similar lines is vast, for example, including behavior of integer, such as Großschädl et al. (2010) and floating-point, such as Andryscio et al. (2015) and Kohlbrenner and Shacham (2017), instruction execution, plus wider, system-level components such as the C compiler (Kaufmann et al. 2016), the JIT compiler (Brennan et al. 2020) and the garbage collector (Pedersen and Askarov 2017).

Note that this chapter intentionally avoids coverage of so-called micro-architectural attacks, many of which are based on data-dependent execution latency in some way: Chapter 2 offers detailed coverage, but, for example, various related books such as Rebeiro et al. (2015) and surveys such as Ge et al. (2018) and Szefer (2019) may also be interesting. Also, although we adopted a focus on cryptographic examples, it is vital to stress the applicability of the same underlying concepts in other contexts: examples include recovery of key-stroke information (Song et al. 2001; Tey et al. 2013; Lipp et al. 2017; Balagani et al. 2018), recovery of resources associated with web-browsing (Felten and Schneider 2000; Kotcher et al. 2013; van Goethem et al. 2015), and traffic analysis (Murdoch and Danezis 2005; Wang et al. 2005). Likewise, it may be interesting to consider that data-dependent execution latency has *constructive* as well as *destructive* use-cases: examples include IP protection (Donda et al. 2015).

– Section 1.2.1. Among numerous explanatory examples, of side-channel attacks in general and those specifically based on execution latency, the presentation by Naccache and Tunstall (2000) is an often-cited instance. Although not strictly comparable to MATCH, it is interesting to note that the manual page for the C `memcmp` function warns “[d]o not use `memcmp()` to compare security critical data, such as cryptographic secrets, because the required CPU time depends on the number of equal bytes”, and so hints at similar real-world behavior.

– Section 1.2.2. The attack description follows the study that of Koeune and Quisquater (1999), which predates the standardization of Advanced Encryption Standard (AES); although compatible in the scenario used, the authors consider then AES candidate Rijndael.

– Section 1.2.3. Rivest et al. (1978) introduced what we now know as the RSA public-key encryption scheme. This attack demands careful understanding of the associated implementation. As such, Gordon (1998) offers a general overview of exponentiation algorithms; Montgomery (1985) describes the eponymous Montgomery reduction technique, with Koç et al. (1996) surveying various implementation options for it. The attack strategy was already outlined by Kocher (1996), with further analysis later offered by Dhem et al. (1998). The attack description follows the latter, focusing on the second case they consider in Dhem et al. (1998), section 3.4, that is, attacking the modular squaring. Note that similar attacks are possible for other scenarios: examples include Mittmann and Schindler (2021), who consider Barrett (1987) reduction, and Bakhshi and Sadeghiyan (2007), who consider Blakley (1983) reduction.

– Section 1.2.4. This attack demands careful understanding of the associated implementation. As such, note that the components involved are all standardized in some way: see, for example, AES (2001), CBC mode (National Institute of Standards and Technology (NIST) 2001b, section 6.2), HMAC (2008) and the TLS-like padding scheme assumed (Dierks and Rescorla 2008, section 6.2.3.2). Set within a symmetric-focused scenario, the attack description follows that of Vaudenay (2002) (later generalized by Black and Urtubia 2002). A wide range of work has applied this approach to real-world targets: examples include Canvel et al. (2003) and Paterson and Yau (2004) and Yau et al. (2005), who consider it as applied to TLS- and ISO/IEC-based cases, respectively, Avoine and Ferreira (2018), who consider padding oracle attacks on a SCP02-compliant smart-card, and both Lucky 13 (AlFardan and Paterson 2013) and Lucky Microseconds (Albrecht and Paterson 2016), which represent further attacks on CBC-based encryption in (D)TLS. It is also important to acknowledge conceptually analogous alternatives set within an asymmetric-focused scenario: important examples include the attacks due to Bleichenbacher (1998) (see also Jager et al. (2012); Meyer et al. (2014); Böck et al. (2018); Ronen et al. (2019)) and later Manger (2001) (see also Strenzke (2010)).

– Section 1.3. Coverage of mitigation, or countermeasures, is challenging in the sense the topic is broad and diverse. From a software perspective only, in order to reduce the scope somewhat, an accessible starting point would be the “coding rules” for implementations maintained by Aumasson Github (n.d.a) and others. At a high level, data-dependent control-flow is supported by numerous, context-dependent approaches. For RSA, relevant examples include various work, for example, Walter (1999) and Hachez and Quisquater (2000), which considers elimination the data-dependent conditional subtraction in Montgomery multiplication. For ECC, relevant examples include use of complete formula, for example, as per Renes et al. (2016), and regular scalar multiplication based on the Montgomery ladder (Montgomery 1987) (thoroughly investigated by Joye and Yen (2003)); Joye (2004) and Oswald (2004) offer a general overview from destructive and constructive perspectives. At a low level, Molnar et al. (2006) propose the program counter

security model, which is essentially a way to formally reason about data-dependent control-flow; such reasoning includes both checking for data-(in)dependence, and transformation of instruction sequences. Coppens et al. (2009) cover a range of related techniques for, for example, control-flow linearization, automation within an LLVM-based compiler back-end for x86 platforms.

It seems fair to say that, at least at the time of writing, tooling which supports (semi-)automatic analysis of implementations is limited but improving. As part of a wider survey, Barbosa et al. (2021), section IV present an overview; relevant examples can be categorized into static, for example, *ct-verif* (Almeida et al. 2016), *FlowTracker* (Rodrigues et al. 2016), *MicroWalk* (Wichelmann et al. 2018), *SideTrail* (Athanasίου et al. 2018) and *FaCT* (Cauligi et al. 2019) and dynamic, for example, *dudec* (Reparaz et al. 2017), *ctgrind* (Github n.d.b), *Triggerflow* (Gridin and Brumley 2019) and *DATA* (Weiser et al. 2018, 2020) cases.

1.5. References

- Aciiçmez, O., Schindler, W., Koç, Ç. (2005). Improving Brumley and Boneh timing attack on unprotected SSL implementations. In *CCS '05: Proceedings of the 12th ACM Conference on Computer and Communications Security*, 139–146.
- Albrecht, M.R. and Paterson, K.G. (2016). Lucky microseconds: A timing attack on amazon's s2n implementation of TLS. In *35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Vienna, 622–643.
- AlFardan, N.J. and Paterson, K.G. (2013). Lucky thirteen: Breaking the TLS and DTLS record protocols. In *2013 IEEE Symposium on Security and Privacy*, Berkeley, CA, 526–540.
- Almeida, J.B., Barbosa, M., Barthe, G., Dupressoir, F., Emmi, M. (2016). Verifying constant-time implementations. In *Proceedings of the 25th USENIX Security Symposium*, Austin, 53–70.
- Anderson, R. (2020). Side channels. In *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd edition. Anderson, R. (edition). John Wiley & Sons, New York. doi: 10.5555/1373319.
- Andryscio, M., Kohlbrenner, D., Mowery, K., Jhala, R., Lerner, S., Shacham, H. (2015). On subnormal floating point and abnormal timing. In *IEEE Symposium on Security and Privacy*, 623–639.
- Athanasίου, K., Cook, B., Emmi, M., MacCarthaigh, C., Schwartz-Narbonne, D., Tasiran, S. (2018). SideTrail: Verifying time-balancing of cryptosystems. In *Verified Software. Theories, Tools and Experiments (VSTTE)*. Springer, Cham. doi: 10.1007/978-3-030-03592-1_12.
- Avoine, G. and Ferreira, L. (2018). Attacking GlobalPlatform SCP02-compliant smart cards using a padding oracle attack. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(2), 149–170 [Online]. Available at: <https://tches.iacr.org/index.php/TCHES/article/view/878>.
- Bakhshi, B. and Sadeghiyan, B. (2007). A timing attack on Blakley's modular multiplication algorithm, and applications to DSA. *Applied Cryptography and Network Security*, 129–140.

- Balagani, K.S., Conti, M., Gasti, P., Georgiev, M., Gurtler, T., Lain, D., Miller, C., Molas, K., Samarin, N., Saraci, E. et al. (2018). SILK-TV: Secret information leakage from keystroke timing videos. In *European Symposium on Research in Computer Security*, 263–280.
- Barbosa, M., Barthe, G., Bhargavan, K., Blanchet, B., Cremers, C., Liao, K., Parno, B. (2021). SoK: Computer-aided cryptography. In *IEEE Symposium on Security and Privacy*, San Francisco. doi: 10.1109/SP40001.2021.00008.
- Barrett, P. (1987). Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor. In *Advances in Cryptology*, 311–323.
- Black, J. and Urtubia, H. (2002). Side-channel attacks on symmetric encryption schemes: The case for authenticated encryption. In *Proceedings of the 11th USENIX Security Symposium*, 327–338.
- Blake, I., Seroussi, G., Smart, N. (eds) (2004). *Advances in Elliptic Curve Cryptography*, 1st edition. Cambridge University Press, Cambridge. doi: 10.1017/CBO9780511546570.
- Blakley, G. (1983). A computer algorithm for calculating the product AB modulo M . *IEEE Transactions on Computers*, 32(5), 497–500. doi: 10.1109/TC.1983.1676262.
- Bleichenbacher, D. (1998). Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1. In *Advances in Cryptology – CRYPTO '98*, 1–12.
- Böck, H., Somorovsky, J., Young, C. (2018). Return of Bleichenbacher's oracle threat (ROBOT). In *27th Usenix Security Symposium*, 817–849.
- Brennan, T., Rosner, N., Bultan, T. (2020). JIT leaks: Inducing timing side channels through just-in-time compilation. In *Proceedings - 2020 IEEE Symposium on Security and Privacy*, 1207–1222.
- Brumley, D. and Boneh, D. (2003). Remote timing attacks are practical. In *12th Usenix Security Symposium*.
- Brumley, B.B. and Tuveri, N. (2011). Remote timing attacks are still practical. In *European Symposium on Research in Computer Security*, 355–371.
- Canvel, B., Hiltgen, A.P., Vaudenay, S., Vuagnoux, M. (2003). Password interception in a SSL/TLS channel. In *Annual International Cryptology Conference*, 583–599.
- Cauligi, S., Soeller, G., Johannesmeyer, B., Brown, F., Wahby, R., Renner, J., Grégoire, B., Barthe, G., Jhala, R., Stefan, D. (2019). FaCT: A DSL for timing-sensitive computation. In *Programming Language Design and Implementation (PLDI)*, June 22–26. ACM, Phoenix. doi: 10.1145/3314221.3314605.
- Coppens, B., Verbauwhede, I., De Bosschere, K., De Sutter, B. (2009). Practical mitigations for timing-based side-channel attacks on modern x86 processors. In *Proceedings of the IEEE Symposium on Security and Privacy*, 45–60.
- Dhem, J.-F., Koeune, F., Leroux, P.-A., Mestré, P., Quisquater, J.-J., Willems, J.-L. (1998). A practical implementation of the timing attack. In *Smart Card Research and Applications (CARDIS)*. Springer, Berlin, Heidelberg. doi: 10.1007/10721064_15.
- Dierks, T. and Rescorla, E. (2008). The Transport Layer Security (TLS) protocol version 1.2. Internet Engineering Task Force (IETF) Request for Comments (RFC) 5246 [Online]. Available at: <http://tools.ietf.org/html/rfc5246>.
- Donda, A.-T., Samarin, P., Samotyja, J., Lemke-Rust, K., Paar, C. (2015). Remote IP protection using timing channels. In *International Conference on Information Security and Cryptology*,

222–237.

- Felten, E.W. and Schneider, M.A. (2000). Timing attacks on web privacy. In *Proceedings of the ACM Conference on Computer and Communications Security*, 25–32.
- Ge, Q., Yarom, Y., Cock, D., Heiser, G. (2018).. A survey of microarchitectural timing attacks and countermeasures on contemporary hardware. *Journal of Cryptographic Engineering (JCEN)*, 8, 1–27. doi: 10.1007/s13389-016-0141-6.
- Github (n.d.a). Cryptocoding [Online]. Available at: <https://github.com/veorq/cryptocoding>.
- Github (n.d.b). Checking that functions are constant time with Valgrind [Online]. Available at: <https://github.com/agl/ctgrind>.
- van Goethem, T., Joosen, W., Nikiforakis, N. (2015). The clock is still ticking: Timing attacks in the modern web. In *CCS '15: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 1382–1393.
- Gordon, D. (1998). A survey of fast exponentiation methods. *Journal of Algorithms*, 27, 129–146. doi: 10.1006/jagm.1997.0913.
- Gridin, I. and Brumley, C.G.N.T.B. (2019). Triggerflow: Regression testing by advanced execution path inspection. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, Cham. doi: 10.1007/978-3-030-22038-9_16.
- Großschädl, J., Oswald, E., Page, D., Tunstall, M. (2010). Side-channel analysis of cryptographic software via early-terminating multiplications. In *International Conference on Information Security and Cryptology*, 176–192.
- Hachez, G. and Quisquater, J.-J. (2000). Montgomery exponentiation with no final subtractions: Improved results. In *International Workshop on Cryptographic Hardware and Embedded Systems*, 293–301.
- Jager, T., Schinzel, S., Somorovsky, J. (2012). Bleichenbacher's attack strikes again: Breaking PKCS#1 v1.5 in XML encryption. In *European Symposium on Research in Computer Security*, 752–769.
- Joye, M. (2004). Defences against side-channel analysis. In *Advances in Elliptic Curve Cryptography*. Cambridge University Press, Cambridge. doi: 10.1017/CBO9780511546570.
- Joye, M. and Yen, S.-M. (2003). The Montgomery powering ladder. In *International Workshop on Cryptographic Hardware and Embedded Systems*, 291–302.
- Kaufmann, T., Pelletier, H., Vaudenay, S., Villegas, K. (2016). When constant-time source yields variable-time binary: Exploiting Curve25519-donna built with MSVC 2015. In *Cryptology and Network Security: 15th International Conference*, 573–582.
- Koç, Ç.K., Acar, T., Kaliski, B. (1996). Analyzing and comparing Montgomery multiplication algorithms. *IEEE Micro*, 16(3), 26–33. doi: 10.1109/40.502403.
- Kocher, P.C. (1996). Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In *Annual International Cryptology Conference*, 104–113.
- Koeune, F. (2011). *Timing Attack*, 2nd edition. Springer, New York. doi: 10.1007/978-1-4419-5906-5.
- Koeune, F. and Quisquater, J.-J. (1999). A timing attack against Rijndael. Technical Report CG-1999/1, Université catholique de Louvain.

- Kohlbrenner, D. and Shacham, H. (2017). On the effectiveness of mitigations against floating-point timing channels. In *26th Usenix Security Symposium*, 69–81.
- Kotcher, R., Pei, Y., Jumde, P., Jackson, C. (2013). Cross-origin pixel stealing: Timing attacks using CSS filters. In *CCS '13: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*, 1055–1062.
- Lipp, M., Gruss, D., Schwarz, M., Bidner, D., Maurice, C., Mangard, S. (2017). Practical keystroke timing attacks in sandboxed JavaScript. In *22nd European Symposium on Research in Computer Security, Proceedings*. Springer-Verlag, Italy, 191–209.
- Manger, J. (2001). A chosen ciphertext attack on RSA optimal asymmetric encryption padding (OAEP) as standardized in PKCS #1 v2.0. In *Annual International Cryptology Conference*, 230–238.
- Meyer, C., Somorovsky, J., Weiss, E., Schwenk, J., Schinzel, S., Tews, E. (2014). Revisiting SSL/TLS implementations: New Bleichenbacher side channels and attacks. In *Proceedings of the 23rd USENIX Security Symposium*, San Diego, 733–748.
- Mittmann, J. and Schindler, W. (2021). Timing attacks and local timing attacks against Barrett's modular multiplication algorithm. *Journal of Cryptographic Engineering (JCEN)*, 11(4), 369–397. doi: 10.1007/s13389-020-00254-3.
- Molnar, D., Piotrowski, M., Schultz, D., Wagner, D. (2006). The program counter security model: Automatic detection and removal of control-flow side channel attacks. In *International Conference on Information Security and Cryptology*, 156–168.
- Montgomery, P. (1985). Modular multiplication without trial division. *Mathematics of Computation*, 44(170), 519–521. doi: 10.1090/S0025-5718-1985-0777282-X.
- Montgomery, P. (1987). Speeding the pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48(177), 243–264. doi: 10.1090/S0025-5718-1987-0866113-7.
- Murdoch, S.J. and Danezis, G. (2005). Low-cost traffic analysis of tor. In *2005 IEEE Symposium on Security and Privacy*, 183–195.
- Naccache, D. and Tunstall, M. (2000). How to explain side-channel leakage to your kids (invited talk). In *CHES 2000*, 229–230.
- National Institute of Standards and Technology (NIST) (2001a). Advanced Encryption Standard (AES). Federal Information Processing Standard (FIPS) 197 [Online]. Available at: <http://csrc.nist.gov>.
- National Institute of Standards and Technology (NIST) (2001b). Recommendation for Block Cipher Modes of Operation: Methods and Techniques. Special Publication 800-38A [Online]. Available at: <http://csrc.nist.gov>.
- National Institute of Standards and Technology (NIST) (2008). The Keyed-Hash Message Authentication Code (HMAC). Federal Information Processing Standard (FIPS) 198 [Online]. Available at: <http://csrc.nist.gov>.
- Oswald, E. (2004). Side-channel analysis. In *Advances in Elliptic Curve Cryptography*, 1st edition. Cambridge University Press, Cambridge. doi: 10.1017/CBO9780511546570.
- Paterson, K.G. and Yau, A. (2004). Padding oracle attacks on the ISO CBC mode encryption standard. In *Cryptographers' Track at the RSA Conference*, 305–323.

- Pedersen, M.V. and Askarov, A. (2017). From trash to treasure: Timing-sensitive garbage collection. In *2017 IEEE Symposium on Security and Privacy, SP 2017 – Proceedings*, 693–709.
- Rebeiro, C., Mukhopadhyay, D., Bhattacharya, S. (2015). *Timing Channels in Cryptography*. Springer, Cham. doi: 10.1007/978-3-319-12370-7.
- Renes, J., Costello, C., Batina, L. (2016). Complete addition formulas for prime order elliptic curves. In *Proceedings, Part I, of the 35th Annual International Conference on Advances in Cryptology – EUROCRYPT 2016*, 403–428.
- Reparaz, O., Balasch, J., Verbauwhede, I. (2017). Dude, is my code constant time? In *Design, Automation & Test in Europe (DATE)*. IEEE, Lausanne. doi: 10.23919/DATE.2017.7927267.
- Rivest, R.L., Shamir, A., Adleman, L.M. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120–126.
- Rodrigues, B., Fernando, M., Aranha, D. (2016). Sparse representation of implicit flows with applications to side-channel detection. In *International Conference on Compiler Construction (CC)*, ACM, New York, 110–120. doi: 10.1145/2892208.2892230.
- Ronen, E., Gillham, R., Genkin, D., Shamir, A., Wong, D., Yarom, Y. (2019). The 9 lives of Bleichenbacher's CAT: New cache ATtacks on TLS implementations. In *2019 IEEE Symposium on Security and Privacy (SP)*, 435–452.
- Schindler, W. (2000). A timing attack against RSA with the Chinese remainder theorem. In *International Workshop on Cryptographic Hardware and Embedded Systems*, 109–124.
- Song, D.X., Wagner, D.A., Tian, X. (2001). Timing analysis of keystrokes and timing attacks on SSH. In *10th Usenix Security Symposium*, Washington, D.C.
- Strenzke, F. (2010). Manger's attack revisited. In *International Conference on Information and Communications Security*, 31–45.
- Szefer, J. (2019). Survey of microarchitectural side and covert channels, attacks, and defences. *Journal of Hardware and Systems Security*, 3(3), 219–234. doi: 10.1007/s41635-018-0046-1.
- Tey, C.M., Gupta, P., Gao, D., Zhang, Y. (2013). Keystroke timing analysis of on-the-fly web apps. In *International Conference on Applied Cryptography and Network Security*, 405–413.
- Vaudenay, S. (2002). Security flaws induced by CBC padding – Applications to SSL, IPSEC, WTLS... In *EUROCRYPT '02: Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques: Advances in Cryptology*, 534–546.
- Walter, C. (1999). Montgomery exponentiation needs no final subtractions. *IEE Electronics Letters*, 35(21), 1831–1832. doi: 10.1049/el:19991230.
- Wang, X., Chen, S., Jajodia, S. (2005). Tracking anonymous peer-to-peer VoIP calls on the Internet. In *CCS '05: Proceedings of the 12th ACM Conference on Computer and Communications Security*, 81–91.
- Weiser, S., Zankl, A., Spreitzer, R., Miller, K., Mangard, S., Sigl, G. (2018). DATA – differential address trace analysis: Finding address-based side-channels in binaries. In *27th Usenix Security Symposium*, 603–620.

- Weiser, S., Schrammel, D., Bodner, L., Spreitzer, R. (2020). Big numbers – Big troubles: Systematically analyzing nonce leakage in (EC)DSA implementations. In *29th Usenix Security Symposium*, 1767–1784.
- Wichelmann, J., Moghimi, A., Eisenbarth, T., Sunar, B. (2018). MicroWalk: A framework for finding side channels in binaries. In *Annual Computer Security Applications Conference (ACSAC)*. doi: 10.1145/3274694.3274741.
- Yau, A.K.L., Paterson, K.G., Mitchell, C.J. (2005). Padding oracle attacks on CBC-mode encryption with secret and random IVs. In *International Workshop on Fast Software Encryption*, 299–319.

2

Microarchitectural Attacks

Yuval YAROM

Ruhr University Bochum, Germany

In this chapter, we investigate a class of side-channel attacks that exploit the *microarchitecture* of the processor. Microarchitecture is the name for the set of components in the processor that implement the functionality it provides, that is, execute the instruction set. When multiple programs run on the same computer, they share the use of microarchitectural components. Such sharing is supposed to be transparent to the programs. However, because the state of many of these components affects execution speed, monitoring execution speed may leak information from one program to another. Microarchitectural side-channel attacks develop techniques for identifying and observing such leaked information.

2.1. Background

We first look at the main microarchitectural components and discuss how they interact with the software and improve its performance. We start with a discussion of the various caches that processors employ and continue to speculative, out-of-order execution.

2.1.1. *Memory caches*

Caches are small banks of fast memory that store recent results in order to provide them quickly if these results are needed again. The main caches in the processor store data retrieved from memory, exploiting the spatial and temporal locality that software often exhibits.

Embedded Cryptography 1,

coordinated by Emmanuel PROUFF, Guénaél RENAULT,
Matthieu RIVAIN and Colin O'FLYNN. © ISTE Ltd 2025.

In operation, the cache divides the address space into blocks, called *lines*, of a fixed size, which on most modern processors are 64 bytes. When the program requests memory, for example, using a load instruction, the processor searches the cache to see if the line that contains the requested address is in the cache. In a *cache hit*, when the line is cached, the processor uses the cached data. Conversely, in a *cache miss*, when the line is not cached, the processor needs to bring the data from the memory. In that case, the line is typically also stored in the cache for future use. Because the cache has a limited size, eventually some lines need to be evicted from the cache to make room for storing new lines. Most caches employ a variant of least recently used (LRU) replacement policy to decide which line to evict.

Modern processors use *set associative* caches, meaning that the cache is divided into a number of *sets*, where each set consists of a fixed number of *ways*. The number of ways in each set is also referred to as the cache associativity. Each memory line can be stored in only one of the sets of the cache, but can be stored in any of the ways of the cache set. In many cases, to map a memory line to a cache set, the processor just uses a sequence of bits from the address of the memory line as an index to the cache. However, some processors use more complex functions to map addresses to cache sets.

2.1.2. Cache hierarchies

To cater for multiple trade offs between cache speed and size, modern processors employ a hierarchy of caches, with caches at the top of the hierarchy (closer to the processor) being smaller but faster than caches lower in the hierarchy. Figure 2.1 shows an example of the cache hierarchy of a four-core Intel Core processor.

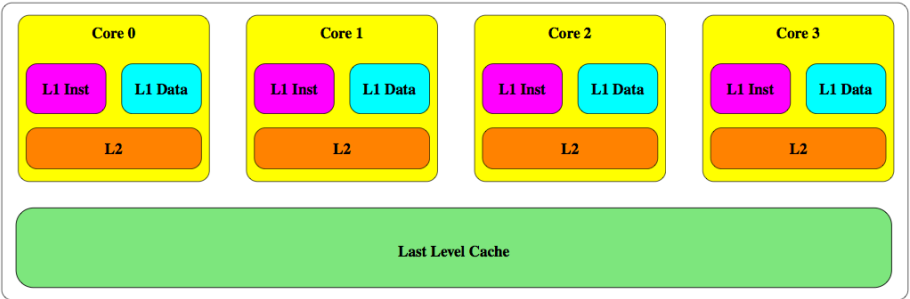


Figure 2.1. The typical cache hierarchy of a four-core Intel processor. For a color version of this figure, see www.iste.co.uk/prouff/cryptography1.zip

Each execution core includes three caches. Two L1 caches, one for storing data and the other for instructions (code), and an L2 cache, which is unified, that is, stores both

Copyright © 2025, John Wiley & Sons, Incorporated. All rights reserved.

data and instructions. All cores share a unified last level cache (LLC), whose size is several megabytes (the exact size depends on the processor model). The LLC in many Intel processor models is *inclusive*. That is, the contents of the LLC is a superset of the contents of the upper level caches. One implication of an inclusive cache is that when a cache line is evicted from the LLC, it must also be evicted from all of the L1 and L2 caches. Table 2.1 shows typical parameters for the caches in Intel processors.

Cache	Number of Sets	Associativity	Total Size	Latency
L1-D	64	8	32 KB	4 cycles
L1-I	64	8	32 KB	4 cycles
L2	512	8	256 KB	7 cycles
LLC	From 4096	12 or more	From 3 MB	28–32 cycles

Table 2.1. Typical cache parameters for many Intel Core processors

2.1.3. Out-of-order execution

To increase the utilization of the processor units, processors do not execute instructions in the order specified in the program. Instead, they can change the order of execution, as long as they ensure that the change does not affect the computation results. The processor core is typically divided into two main parts. The *front end* is responsible for reading the code of the program from memory, parsing it into instructions, and decoding them into a format that the processor uses for execution. In some processors, decoding includes translation into microcode, a set of elementary operations, called μ ops. In this chapter, we use the term “instruction” for both instructions and μ ops, ignoring the distinction between the two.

The *execution engine* receives a stream of instructions from the front end and routes them to the various execution units. The execution engine dispatches instructions for execution as soon as all of its arguments are available. Consequently, a young instruction can execute before an older instruction, that precedes it in the instruction stream, if the arguments of the younger instruction become available before those of the older instruction. The execution engine notifies the front end when instructions complete execution. The front end then retires instructions, committing their results to the state of the program. Instructions are only retired in their order in the program, maintaining the illusion of in-order execution.

Special handling is required when an instruction results in a trap, such as division by zero or illegal access to memory. To maintain the illusion of in-order execution, instead of handling the trap as soon as it is detected, the execution engine notifies the front end that the instruction causes a trap. The front end waits until all older instructions retire before handling the trap. It then squashes all instructions younger than the trapped instruction, ignoring any results they may have computed, and finally

handles the trap. Instructions that are squashed after they were executed are often called transient instructions.

2.1.4. Branch prediction

Branches pose a challenge to out-of-order execution. If it takes time to compute the outcome of a branch, it may be desirable to start executing younger instructions that can be executed. However, before computing the branch outcome, the processor does not know where processing follows.

To address this challenge, when the front end encounters a branch, it tries to predict the outcome of the branch and proceed to process younger instructions based on the predicted outcome. Eventually, the execution engine executes the branch and compares the outcome to the prediction. If the prediction turns out to be correct, the program benefits from the early computation of younger instructions. Conversely, if the prediction turns out to be wrong, the front end squashes all mispredicted instructions and proceeds to issue instructions from the correct branch.

To predict branch outcomes, the processor maintains an array of dedicated caches that store the outcomes of recent branches. The two main caches are the branch target buffer (BTB) that stores the addresses that the branches branch to, aiming to predict indirect branches, and the branch history buffer (BHB) that predicts whether conditional branches will be taken or not.

2.1.5. Other caches

In addition to the main memory caches and the caches used for branch prediction, processors typically feature other caches. The translation lookaside buffer (TLB) is a dedicated cache that stores the results of recent translations of virtual addresses to physical addresses. The microcode cache is a front-end component that stores the results of recent translations of instructions to μ ops, reducing the overhead of such translation. Caching elements also exist outside the processor. For example, the row buffer in dynamic random access memory (DRAM) chips stores the contents of the most recently accessed memory row.

2.2. The Prime+Probe attack

This section discusses Prime+Probe, an attack technique that exploits the structure of the cache to leak information between processes that share the use of the cache. The attack consists of two main steps. In the prime step, the attacker completely fills one or more of the sets of the cache with its own data. In the probe step, the attacker measures the time it takes to access the data it previously used for filling the cache.

Between the two steps, the attacker gives the victim the opportunity to run and access memory. If the memory that the victim access is mapped to any of the cache sets that the attacker uses for the attack, victim access will evict some of the attacker's data. Hence, access during the probe step will incur delays due to cache misses, allowing the attacker to identify the cache sets that the victim has accessed.

2.2.1. Prime+Probe on the L1 data cache

As an example of a concrete Prime+Probe attack, we target the L1 data cache of a typical Intel Core processor. The L1 cache in these processors consists of 64 sets, each containing 8 or 12 cache lines, depending on the processor model. The size of each cache line is 64 bytes. To determine the cache set number that a memory address maps to, the processor uses bits 6–11 of the address.

For the rest of the example, we will assume that the L1 data cache is eight-way associative, that is, it has eight cache lines in each cache set. To fill a cache set, we need to find eight cache lines that map to it. A simple way to achieve that is to allocate a page-aligned cache-sized (32 KB) buffer, for example, using the `mmap` system call. Then, given a cache set $0 \leq s < 64$, the eight offsets $64 \cdot s + 4096 \cdot i$ for $0 \leq i < 8$ all map to cache set s . We use the term *eviction set* to refer to such a collection of memory locations that completely cover a cache set.

To prime a cache set, we can now read the contents of the eight memory locations in the eviction set. This will bring the eight locations into the cache and evict any other data from the same cache set. Technically, probing the cache set mainly requires measuring the time to access the same eviction set to check whether it is in the cache. However, due to out-of-order execution, a naive measurement is unlikely to achieve conclusive results.

To handle out-of-order execution, we need to first ensure that measured memory accesses are performed during the measured period. For that we use a fence instruction `lfence` before and after the measured sequence. The `lfence` instruction ensures that no younger instruction starts executing before all the instructions older than the `lfence` are committed. A second issue is that a memory miss delay may be masked due to concurrent, out-of-order execution. To overcome the issue, we need to introduce a dependency between memory accesses. A simple way is to create a linked list where each of the elements in the eviction set points to the next. When traversing the list, the processor has to wait for the data of one memory location before starting a load from the next. Consequently, the memory accesses execute sequentially, and the delay is not masked.

```

1 s0 = GETU32(in      ) ^ rk[0];
2 s1 = GETU32(in + 4) ^ rk[1];
3 s2 = GETU32(in + 8) ^ rk[2];
4 s3 = GETU32(in + 12) ^ rk[3];
5
6 /* round 1: */
7 t0 = Te0[s0>>24] ^ Te1[(s1>>16) & 0xff] ^ Te2[(s2 >>8) & 0xff] ^ Te3[s3 & 0xff] ^ rk[ 4];
8 t1 = Te0[s1>>24] ^ Te1[(s2>>16) & 0xff] ^ Te2[(s3 >>8) & 0xff] ^ Te3[s0 & 0xff] ^ rk[ 5];
9 t2 = Te0[s2>>24] ^ Te1[(s3>>16) & 0xff] ^ Te2[(s0 >>8) & 0xff] ^ Te3[s1 & 0xff] ^ rk[ 6];
10 t3 = Te0[s3>>24] ^ Te1[(s0>>16) & 0xff] ^ Te2[(s1 >>8) & 0xff] ^ Te3[s2 & 0xff] ^ rk[ 7];

```

Listing 2.1. First encryption round of the T-table implementation of AES

2.2.2. Attacking T-table AES

As an example, we will look at attacking an implementation of AES that uses T-tables. To recap, the T-tables in AES combine the effects of the sub-bytes, shift-rows, and mix-columns steps of the encryption. The implementation uses four T-tables, Te0, Te1, Te2 and Te3. Each table consists of 256 entries of size 32 bits each. In each encryption round, the implementation XORs each state byte with the corresponding byte of the round key and uses the result to index one of the four T-tables. Thus, in each of the 10 rounds of AES-128, each T-table is accessed four times, to a total of 40 times over the encryption. Listing 2.1 shows the first round of the encryption. In lines 1–4 the code reads the plaintext and XORs it with the round key (stored in rk). Then, in lines 7–10, the code uses each byte of the state to index one of the four T-tables.

When attacking such an implementation, the adversary tries to learn information about the key by monitoring the cache access patterns. Specifically, the adversary first primes all of the cache sets of the L1 cache. The victim then performs one AES encryption with a known plaintext and an unknown key. Finally, the adversary performs the probe step of the attack to identify the cache sets accessed by the victim.

Clearly, if the adversary can determine which index in the T-tables is read at each step of the algorithm, they can recover the key. However, cache attacks have a limited resolution, both spatially and temporally. Specifically, the Prime+Probe attack has a cache-line resolution and cannot determine where within the cache line an access takes place. In the T-tables, each cache line includes 16 table entries. Consequently, the attacker can at best determine the most significant four bits of the accessed index.

Moreover, performing a cache attack takes time. During that time, the victim may perform several memory accesses. The Prime+Probe attack cannot determine the order of these memory accesses or the number of accesses to the same cache line. In our example, the victim performs a full encryption between the prime and the probe steps. Consequently, the adversary can determine which cache lines in the T-tables have been accessed during the encryption, but not the round in which the access happens.

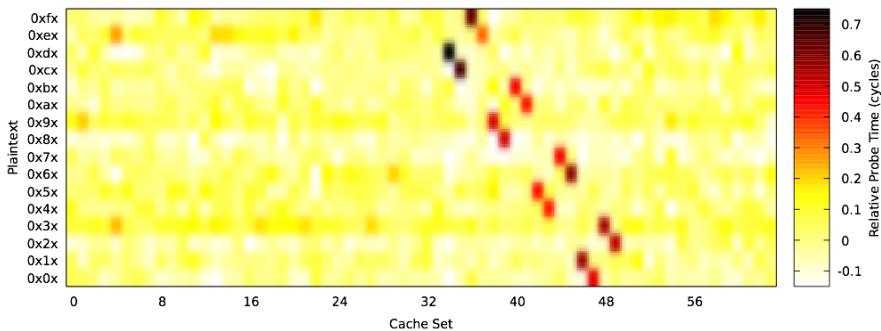


Figure 2.2. Prime+Probe on AES. The shade indicates the relative probe time for cache set (X axis) and plaintext byte value (most significant four bits, Y axis). Darker shades indicate slower probe identifying the XOR of the plaintext and the key. For a color version of this figure, see www.iste.co.uk/prouff/cryptography1.zip

To overcome the limited temporal resolution of Prime+Probe, we note that there is some probability that a cache line is not accessed at all during an encryption. Specifically, each T-table spans 16 cache lines and is accessed 40 times during an encryption. Thus, there is a probability of $\left(\frac{15}{16}\right)^{40} \approx 7.6\%$ that a specific cache line is not accessed at all during the encryption.

At the same time, the accessed indices are not random and are determined by the plaintext and the key. Hence, when encrypting multiple plaintexts that have the same value in one of the plaintext bytes, the same T-table index is accessed during the first round, and the cache line that contains it will always be accessed. Consequently, the average probe time of one specific cache set will be slower when fixing one plaintext byte than when using a random value.

Figure 2.2 shows the result of carrying out the Prime+Probe attack on 100,000 encryptions. The Y axis indicates the value of the four most significant bits of byte 6 of the plaintext. The X axis is the cache set. The shade indicates the difference between the average probe time of the cache set when the byte value has the indicated value in the four MSBs, compared to the average over all byte values.

As we can see, the average access time to most cache sets does not depend on the plaintext byte. However, cache sets 33–49 do show a plaintext-dependent access pattern, where for each group of plaintext values that have the same four MSBs, the probe time for one of the cache sets is slower by roughly 0.5 cycles, on average. We therefore hypothesize that T-table Te2, which is indexed by the 6th byte of the state, maps to these cache sets. By checking the ground truth, we can confirm the hypothesis.

When we check the relationship between the cache set that experiences slower probes and the four MSBs of the plaintext, we observe that plaintexts that start with bits 1101, that is, values 0xd0–0xdf, result in a slower probe of the first cache set that matches T-table Te2. This indicates that the cache set is always accessed when encrypting a plaintext with byte 6 in the range 0xd0–0xdf.

The index for accessing T-table Te2 when processing byte 6 of the state, is computed by XORing byte 6 of the plaintext with byte 6 of the key. Hence, because the encryption always accesses the first cache set of Te2 when the plaintext byte is in the range 0xd0–0xdf, we learn that the corresponding key byte is also in the same range 0xd0–0xdf. Comparing to the ground truth, where the byte is 0xd2, we can verify the data. We note that we could have used any other line in Figure 2.2, and get the same result.

Repeating the process for each of the 16 plaintext bytes, we learn 64 bits. To recover the rest of the key, we need to extend the attack to a second round. The process is quite similar. Specifically, given the known bits of the first round key bits, we can guess the remaining bits and predict which cache line accesses will happen in the second round. Correlating this information with the probe times we see will identify the correct guess and recover the key bits.

2.2.3. Prime+probe on the LLC

Because the L1 cache is not shared between cores, it cannot be used for cross-core attacks. Targeting the shared LLC allows such attacks, but requires more effort than an L1 attack due to the complexity of finding eviction sets. One cause of the issue is that the adversary uses virtual memory addresses, whereas the cache set is determined by the physical address. Bits 6–11, which encode the cache set in the L1, do not change when mapping virtual to physical addresses. However, the LLC has more cache sets than the L1, requiring more than six bits for addressing LLC cache sets. Bits 12 and above are not preserved during address translation. Consequently, the address translation process hides these bits from the attacker. Moreover, while the L1 uses the bits values as the cache set number, the Intel LLC uses an unpublished hash function.

A simple approach for finding eviction sets is to first find a set of candidate memory locations such that accessing all candidates evicts a *witness* memory location from the cache and then trims it to a minimal size. Algorithm 2.1 shows an example of this approach. The algorithm takes two inputs: a witness memory location w and a set of candidates for evicting w . Typically, these candidates will be chosen so that they match the known bits of the physical address of the witness. In lines 2–6, the algorithm expands the potential eviction set e , adding candidates until accessing the contents of e evicts the witness. The algorithm then tests each of the elements of the potential eviction set to check if they are required for evicting the witness. After removing

all nonessential elements from the potential eviction set, only an eviction set for the witness remains.

Algorithm 2.1. Creating an eviction set for a witness line w

input : w : a witness line
input : c : a set of candidate memory lines for an eviction set for w
output: e : an eviction set for w

```

1  $e \leftarrow \{\}$ 
2 foreach  $l \in c$  do
3    $e \leftarrow e \cup \{l\}$ 
4   if  $\text{evicts}(e, w)$  then
5     break
6 end
7 foreach  $l \in e$  do
8   if  $\text{evicts}(e - \{l\}, w)$  then
9      $e \leftarrow e - \{l\}$ 
10 end
11 output  $e$ 

```

The algorithm can be repeated to find all of the eviction sets for the LLC, allowing the adversary to perform cross-core Prime+Probe attacks. It should be noted that the attack only works if the LLC is inclusive. The reason is that evicting a line from a non-inclusive LLC does not necessarily evict it from the private L1 and L2 caches of the victim. Consequently, the attacker cannot observe victim accesses to lines evicted from the LLC. However, some processors use a cache directory to keep track of the coherency state of lines in the private caches of the cores. Evicting the directory entry also evicts the line from the private caches, allowing the attack.

2.2.4. Variants of Prime+Probe

Prime+Abort: Prime+Abort is a variant of the Prime+Probe attack, which exploits a property of the Intel Transactional Synchronization Extensions (TSX). In a nutshell, TSX is an extension of the x86 instruction set that supports sequences of instructions, called transactions, that are executed atomically. TSX relies on cached data to detect conflicting memory accesses during a transaction. Consequently, eviction of memory accessed during a transaction aborts the transaction.

In a Prime+Abort attack, the attacker starts a TSX transaction and accesses the elements of the eviction set of a cache set, completely filling the cache set. If the victim accesses memory in that cache set, the transaction aborts. The attacker detects the transaction abort and verifies that the abort cause is contention on resources to identify the victim access.

Prime+Abort provides two main advantages over the standard Prime+Probe attack. First, it does not rely on timing, reducing potential measurement errors. Second, unlike Prime+Probe, Prime+Abort does not poll the monitored cache set. Consequently, it achieves a higher temporal resolution. At the same time, Prime+Abort does have some limitations. In particular, the technique is unable to detect which of multiple cache sets is accessed. Moreover, due to security issues, TSX has been disabled in recent processors. Hence, the attack only works on older, unpatched processors.

Prime+Scope: the Prime+Scope attack takes advantage of the interplay between different levels of the cache hierarchy to improve the temporal resolution of the attack. The core idea in Prime+Scope is that when evicting a line from an inclusive LLC or from a cache directory, the line is also evicted from the attacker's L1 cache.

In the prime stage, the attacker fills the LLC cache set, which it monitors, with the contents of the eviction set. Instead of accessing the eviction set in an arbitrary order, the attacker uses a specific access pattern. The pattern is selected so that the LLC replacement candidate, that is, the cache line to be evicted next from the LLC, is also cached in the attacker's L1 cache. For the scope stage, the attacker repeatedly measures the access time to the LLC replacement candidate. Because the LLC replacement candidate is in the L1 cache, the access is fast. Moreover, because the access results in a cache hit, the L1 cache serves the access without notifying the LLC. Hence, the access does not affect the LLC replacement policy, and the LLC does not change the replacement candidate.

When the victim accesses a memory location that maps to the monitored cache set, the LLC needs to evict one of the cached lines and chooses the replacement candidate. Evicting the replacement candidate from the LLC also triggers an eviction from the attacker's L1 cache. Consequently, when the attacker next measures access time to this cache line, it experiences a cache miss in all cache levels, which results in a long access time, indicating a victim access. To resume the attack, the attacker now needs to reset the cache state by repeating the prime stage.

In the absence of victim access to the cache set, the scope stage of Prime+Scope only causes cache hits in the L1 cache. These are much faster than the accesses required to probe the cache set in Prime+Probe, allowing for an order of magnitude improvement in the temporal resolution of Prime+Scope compared to Prime+Probe.

Cache Occupancy Attack: instead of targeting specific cache sets, the cache occupancy variant of the Prime+Probe attack aims to measure the overall contention on the cache. For the attack, the attacker allocates a cache-sized buffer in memory and repeatedly measures the access time to all of the cache lines in the buffer. Each such measurement brings the whole buffer into the cache. Hence, the time to access the buffer correlates with the number of buffer cache lines evicted from the cache

between consecutive measurements. When the victim performs more memory accesses, more cache lines will be evicted, and the measurement will be longer.

With the typical LLC size, each measurement takes hundreds and even thousands of microseconds. Consequently, the temporal resolution is significantly coarser than that of the Prime+Probe attack. Moreover, because the measurement covers the cache, the attacker does not learn about the addresses that the victim accesses, but only about the number of cache lines that the victim has accessed. Thus, the cache occupancy attack is typically used for coarse-grained events, such as detecting running applications and visited web sites. However, cache occupancy attacks against cryptographic schemes have also been demonstrated.

Due to its simplicity, the cache occupancy attack is very resilient. The attack can work in environments, such as web browsers, where the clock resolution is low. It can even work with alternative methods for measuring time, for example, using network latency. Furthermore, because the attack does not rely on a specific cache structure, it can overcome approaches for secure caches, designed to thwart the Prime+Probe attack.

2.3. The Flush+Reload attack

In this section, we discuss the Flush+Reload attack. Unlike Prime+Probe, which exploits the limited storage space in the cache, Flush+Reload exploits the adversary's ability to evict the cache lines it shares with the victim from the cache. Because it focuses on a specific cache line, rather than a cache set, the attack tends to be less noisy than Prime+Probe. However, the requirement for sharing memory limits the scenarios in which the attack is applicable.

2.3.1. Attack technique

To reduce the memory signature of the system, modern operating systems share memory between processes. One form of sharing is when an operating system maps the same library or program code to multiple processes. Another form of sharing is when an operating system or a virtual machine hypervisor coalesces pages with identical contents. As long as none of the sharing programs changes the contents of the shared pages, sharing reduces the combined memory requirements without affecting correctness. In the case of coalescing identical pages, the system typically uses copy-on-write semantics, where the system intercepts attempts by either process to modify the shared page and duplicates the contents to prevent modifications.

The cache does not distinguish between programs. Consequently, if one of the sharing processes accesses a shared memory line, the other process can experience a

cache hit the first time it accesses the line. Conversely if a process evicts a shared line from the cache, it can cause a cache miss for the other process.

The Flush+Reload exploits this sharing of cache state. In the flush step, the adversary evicts a cache line from the cache, for example, using the x86 `clflush` instruction. After allowing the victim some time to execute, in the reload step the adversary measures the time it takes to load the line. If the victim has not accessed the line between the flush and the reload steps, the adversary witnesses a long reload time due to a cache miss. Otherwise, if the victim has accessed the line, the line is cached, and the reload time is shorter.

2.3.2. Attacking square-and-multiply exponentiation

As an example of using a Flush+Reload attack, we now use it against a victim that uses modular exponentiation, a core operation in multiple encryption schemes, such as RSA and ElGamal. Modular exponentiation takes three inputs: a base b , a modulus m and a secret exponent d . Its output is $b^d \bmod m$.

Algorithm 2.2. The square-and-multiply algorithm for modular exponentiation

input : base b , modulus m , and secret exponent $d = \sum_{i=0}^{n-1} d_i 2^i$
output: $b^d \bmod m$

```

1  $x \leftarrow 1$ 
2 for  $i = n - 1$  downto 0 do
3    $x \leftarrow x^2 \bmod m$ 
4   if  $d_i = 1$  then
5      $x \leftarrow x \cdot b \bmod m$ 
6 end
7 return  $x$ 
```

Algorithm 2.2 shows the square-and-multiply algorithm for performing modular exponentiation. The algorithm uses the representation of the exponent as an n -bit binary number, that is, $d = \sum_{i=1}^{n-1} d_i 2^i$, where $d_i \in \{0, 1\}$. It scans the exponent from the most significant to the least significant bit. For each bit, the algorithm squares an accumulator x and if the bit is set it multiplies the accumulator by the base b (all operations calculated modulo the modulus m). At the end of the algorithm, x contains the required result. The algorithm is used in several implementations of RSA and ElGamal encryption, for example, in version 1.4.13 of the GnuPG library.

When using Flush+Reload to attack such an implementation, the adversary monitors the memory lines that contain the code for the square and the multiply functions. That is, the adversary repeatedly flushes the two memory lines, waits a bit

and reloads the lines while measuring the time to reload them. Figure 2.3 shows a sequence of such measurements. The X axis is the sample number and the Y axis shows the time it takes to access the monitored line at each sample. Low values indicate that the monitored line is in the cache, i.e. that which the code is executing.

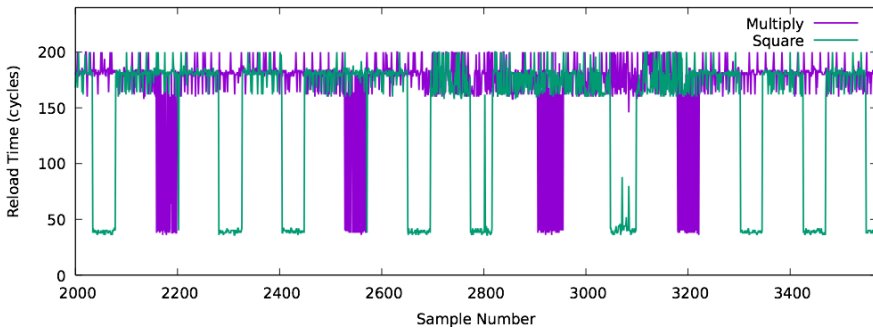


Figure 2.3. *Flush+Reload on the square-and-multiply implementation of modular exponentiation in GnuPG 1.4.13. Square followed by multiply indicates a bit 1, and square followed by another square indicates a bit 0. The exponent bits shown in this example are 10101100. For a color version of this figure, see www.iste.co.uk/prouff/cryptography1.zip*

When the exponent bit is one, the victim executes a square followed by a multiply. When the bit is zero, the victim only executes a square. In Figure 2.3, we see that a square operation took place between samples 2035 and 2079. This was followed by a multiply, at samples 2160–2200. Hence, we can conclude that the exponent bit that the victim processed between samples 2035 and 2200 was set. Conversely, the square operation between samples 2282 and 2327 is followed by another square operation (samples 2406–2449). Hence we conclude that the bit processed between samples 2282 and 2327 is clear. Following this logic, we can read the exponent bits processed in the window presented in Figure 2.3 and find that they are 10101100.

2.3.3. Attack variants

Several variants of the Flush+Reload attack have emerged using other primitives for either evicting the cache or measuring whether the target line is cached. The first such attack, Evict+Reload, targets the scenario of implementing the attack in non-x86 environment, where instructions for direct cache management are privileged and are not available to attack code executing in user-space. Instead of using `clflush` or an equivalent instruction, the adversary can use contention to evict the line from the cache. For that, the attacker needs an eviction set for the cache set in which the target cache line resides. Accessing this eviction set, as done for the prime step of

the Prime+Probe attack, will load the eviction set to the cache, forcing eviction of any conflicting cache line, including the target cache line.

In the Flush+Flush variant, instead of measuring the time it takes to reload the target cache line, the adversary measures the time to flush it again. It turns out that the time it takes an Intel processor to execute the `clflush` instruction depends on whether the target memory line is cached or not. By measuring the flush time, the adversary both determines whether the line is cached and ensures that the line is no longer cached. Because there is no need for a separate reload step, the Flush+Flush attack has a higher temporal resolution than Flush+Reload. That is, the adversary can measure at a higher frequency. Moreover, in the Flush+Flush attack, the adversary does not trigger cache misses. Consequently, it is harder to detect an active Flush+Flush attack by monitoring the cache activity. On the down side, Flush+Flush shows much smaller timing differences than Flush+Reload, and the timing of the `clflush` instruction varies with the core that the attack code executes on. Consequently, the attack is noisier than Flush+Reload and is harder to execute in practice.

Last, the Prefetch+Prefetch attack exploits the cache coherency mechanism to achieve an effect similar to Flush+Reload. Cache coherency ensures that memory writes in one core are visible in all cores of the system. To maintain coherency, write operations in one core evict the target cache line from the private caches of all other cores, ensuring that future accesses in other cores get the updated contents of the cache line. To improve write performance, the x86 architecture provides a `prefetchw` instruction, which evicts a cache line from the private caches of *other* cores, in preparation for a write instruction in the executing core. The implementation of `prefetchw` on Intel processors does not check that the accessing user has write access to the target memory line. Consequently, `prefetchw` evicts the cache line from other cores even when the accessing process is only allowed to read the memory. Moreover, on many Intel processors, the time it takes to execute `prefetchw` depends on the coherence state of the target line. In particular, the time to execute `prefetchw` indicates whether another core has accessed the target memory line between successive invocations of `prefetchw`. Thus, the Prefetch+Prefetch is similar to Flush+Flush.

2.3.4. Performance degradation attacks

The temporal resolution of cache attacks limits the ability of the attacker to observe victim accesses. The attacks are typically performed in rounds that take hundreds to thousands of cycles each. In particular, Flush+Reload presents a trade-off between attack resolution and accuracy, showing a significant drop of accuracy when the time between the flush and the reload steps is less than 2,000 cycles.

A consequence of this limitation is that the attack cannot distinguish between successive victim accesses to the target memory line if the victim access frequency

is higher than the sampling frequency of the attack. Performance degradation attacks aim to slow a victim down to reduce the access frequency and allow the attack to acquire the operation.

The core idea of the attack is to identify the victim code that executes between successive accesses to target memory addresses and flush this victim code from the cache. Repeated flushes of the victim code force the victim to incur a memory access delay when trying to execute the flushed code. The attack can slow down a tight loop by up to three orders of magnitude. For realistic scenarios, the attack typically achieves a slowdown of one to two orders of magnitude, allowing attacks in scenarios that are otherwise almost impossible.

A downside of the attack is that the slowdown is not uniform. Consequently, measurements in the presence of performance degradation tend to be noisier than otherwise.

2.4. Attacking other microarchitectural components

So far, we have looked at attacks that target data caches. In this section, we briefly discuss other microarchitectural components and how they can be exploited for attacks.

2.4.1. *Instruction cache*

Instead of targeting the data caches, an attacker can perform the Prime+Probe attack on one of the instruction caches. The attack typically reveals secret-dependent control flow, rather than secret dependent memory accesses, demonstrated in section 2.2.1.

The main difference between instruction-cache attacks and data-cache attacks is that instead of accessing the eviction sets, the attacker needs to execute code in the eviction set. Listing 2.2 shows an example of x86 code that performs the probe step in a Prime+Probe attack on an Intel L1 cache. The code first aligns the start of the function `L1IProbe()` to a page boundary (line 1). It then takes the current cycle count (line 3), records the clock reading in register `rsi` (line 4) and falls into a sequence of jump instructions that are 4,096 bytes apart. As discussed, the L1 cache has a cache line of 64 bytes and a total of 64 cache sets. Consequently, addresses that are a multiple of 4,096 bytes apart fall within the same L1 cache set. Finally, after visiting eight congruent addresses (for an 8-way cache), the code takes the current cycle count (line 19), subtracts the cycle count taken at the start of the function (line 20) and returns. The code in Listing 2.2 probes cache set 0. To target other cache sets, the attacker needs to offset the code within the page.

```

1  .align 4096
2  L1IProbe:
3      rdtscp
4      mov %rax, %rsi
5      jmp P2
6      .align 4096
7  P2: jmp P3
...
18 .align 4096
19 P8: rdtscp
20     sub %rsi, %rax
21     ret

```

Listing 2.2. Probing the L1 Instruction Cache. For a color version of this listing, see www.iste.co.uk/prouff/cryptography1.zip

Attacks on lower cache levels, such as the attacks in sections 2.2.3 and 2.3 target unified caches that cache both instruction and data. As demonstrated in section 2.3.2, such attacks can recover control flow. The main advantage that attacking the L1 instruction cache has over these attacks is that the temporal resolution is significantly higher. However, the results contain higher levels of noise, presenting a trade-off for the adversary. Moreover, the L1 instruction cache attack can only be applied within the same core, because the cache is private to the core.

2.4.2. Branch prediction

Another approach for retrieving the victim's control flow is to attack the branch predictor. Recall that the processor aims to predict future control flow based on past branch outcomes. The branch predictor itself consists of two main structures: the BTB, which predicts the address of a branch, and the BHB, which predicts whether the branch will be taken or not. On Intel processors, which tend to be popular research targets, conditional branches update the BTB only if the branch is taken. Consequently, attacks on the BTB can recover conditional branch outcomes, reducing the need for attacks on the BHB. Thus, here we focus on attacks that target the BTB.

Like other caches, the BTB is set associative and is therefore vulnerable to Prime+Probe attacks. However, unlike other caches, the BTB does not match the full address. Instead, it uses only some of the address bits (least significant 32 bits in most modern Intel processors). Hence a single BTB entry provides prediction to multiple *aliased* addresses. The main effect of aliasing on attacks is that only one prediction exists for aliased addresses. Hence, if we know the victim branch address, we can evict it from the BTB using an aliased branch, without a need for an eviction set.

2.5. Constant-time programming

The attacks on AES (section 2.2.2) and on modular exponentiation (section 2.3.2) represent the two main sources of leakage through microarchitectural side channels: secret dependent memory accesses and secret dependent control flow. *constant-time programming* is a coding paradigm that protects against such channels by preventing these forms of leakage. It also protects against timing leakage through instructions whose execution time depends on their arguments. This section presents the basics of constant-time programming.

2.5.1. Constant-time select

A core basic block for constant-time programming is the constant-time select operator, which provides a safe alternative to the conditional ternary operator in languages such as C. Specifically, the operator `ct_select(condition, true_val, false_val)` returns the result of the expression `condition ? true_val : false_val`. However, unlike the ternary operator, both values are evaluated, and the selection does not use any conditional control-flow instruction.

A straightforward implementation of `ct_select` is using conditional move instructions, such as the x86 instruction `CMOVcc`. These instructions move the source value to the destination if a condition is true. While efficient, the main drawbacks of these instructions are that they are machine dependent and there is no generic way of forcing their use in languages such as C.

```

1 int ct_select(int condition, int true_val, int false_val) {
2     int mask;
3     mask = condition ^ (condition - 1);
4     mask = mask & ~condition;
5     mask = mask >> (sizeof(int) * CHAR_BIT - 1);
6
7     return (~mask & true_val) | (mask & false_val);
8 }

```

Listing 2.3. An implementation of constant-time select for `int` values. For a color version of this listing, see www.iste.co.uk/prouff/cryptography1.zip

A more generic approach is to use a mask whose value depends on the condition (all zero if the condition is true and all one if the condition is false). We can then calculate the result of `ct_select` using `(~mask & true_val) | (mask & false_val)`. In the case that condition is either 0 or 1, we can set mask to

condition - 1. Listing 2.3 shows an implementation of `ct_select` for the general case, where any nonzero value indicates true. After line 3, the most significant bit of `mask` if `condition` is either zero or `0x80000000` (assuming 32 bit integers). Line 4 clears the most significant bit of `mask` if the most significant bit of `condition` is set. Hence, after line 4, the most significant bit of `mask` is set only if `condition` is 0, that is, false. Line 5 uses arithmetic shift to replicate most significant bit of `mask` to all of the bits, achieving the required value of all zero bits for true condition and all ones otherwise. The code can be adapted to other type sizes, as long as the size of the `mask` matches that of the values.

It should be noted that the code makes two assumptions on the C implementations. First, it assumes that language uses two's complement to represent negative numbers. Moreover, it assumes that the language uses arithmetic right shift for negative numbers. Both assumptions are not stipulated in the standard. Specifically the C (and C++) standards allow other negative number representations and state that right shift of negative numbers is an undefined behavior. However, in practice, the assumptions hold for all mainstream compilers and code similar to Listing 2.3 is common in cryptographic implementations.

2.5.2. Eliminating secret-dependent branches

As we have seen in Algorithm 2.2, when a program executes a branch whose condition depends on secret information, an adversary can use a side-channel attack to determine the condition and infer the secret information. Consequently, to protect against such attacks, constant-time programs avoid secret-dependent branches.

To eliminate secret-dependent branches, the program can always execute both the `then` block and the `else` block of the branch and use constant-time select to choose the required values for the variables. For example, Algorithm 2.3 shows the square-and-multiply-always algorithm, a constant-time version of Algorithm 2.2. As we can see, the `if` statement in the original algorithm (line 4 in Algorithm 2.2) has been removed. Instead, the multiplication is always executed, irrespective of the value of the exponent bit (line 4 Algorithm 2.3). The `ct_select` function in line 5 of Algorithm 2.3 replaces the value of `x` with the multiplication result if the current bit is set.

An important point to note is that `ct_select` should not be used for copying pointers. The reason is that subsequent dereferences of pointers selected with `ct_select` would result in memory accesses where the addresses depend on the secret condition. Instead, in the case of pointers, the program should use `ct_select` to deep-copy the structures that the pointers point to. Specifically, in Algorithm 2.3, the values of `x` and `t` will typically be represented using arrays. For a constant-time implementation, `ct_select` should be used to select each of the elements of the arrays.

Algorithm 2.3. The square-and-multiply-always algorithm for modular exponentiation

input : base b , modulus m , and secret exponent $d = \sum_{i=0}^{n-1} d_i 2^i$
output: $b^d \bmod m$

```

1  $x \leftarrow 1$ 
2 for  $i = n - 1$  downto 0 do
3    $x \leftarrow x^2 \bmod m$ 
4    $t \leftarrow x \cdot b \bmod m$ 
5    $x \leftarrow \text{ct\_select}(d_i = 1, t, x)$ 
6 end
7 return  $x$ 

```

2.5.3. Eliminating secret-dependent memory access

Many implementations of cryptographic primitives include tables, where the choice of element to be accessed depends on secret data. We have seen one such example: the T-tables implementation of AES in Listing 2.1. To harden table access, constant-time programs need to access *all* possible memory addresses and use constant-time select to retain the correct value. Listing 2.4 shows an example of how this could be implemented for T-table access. In the listing, we have a function `ct_ttable` that takes an index `in` and a T-table `T` as arguments. The variable `rv`, which will hold the return value, is first initialized in line 2. Then, the function iterates over all possible values of `in` (line 4) and uses `ct_select` to choose between the previous value of `rv` and the value loaded from the table, based on whether the current index is the same as the input.

```

1 uint32_t ct_ttable(uint8_t in, uint32_t *T) {
2     uint32_t rv = 0ULL;
3
4     for (int i = 0; i < 256; i++)
5         rv = ct_select(i == in, T[i], rv);
6     return rv;
7 }

```

Listing 2.4. An implementation of constant-time T-table access. For a color version of this listing, see www.iste.co.uk/prouff/cryptography1.zip

2.6. Covert channels

So far, we have looked at microarchitectural side-channel attacks, where the attacker monitors the microarchitectural state of the processor to detect unintended leakage of victim execution patterns. Microarchitectural components can also be exploited to implement covert channels. In a covert channel, a *transmitter* (also referred to as a Trojan) seeks to send information to a *receiver* by using a medium that is not designed for communication.

Listing 2.5 shows an example of a cache-based covert channel that exploits the Flush+Reload technique. Specifically, the covert channel uses three functions: `cc_setup()`, `cc_transmit()`, and `cc_receive()`, to manipulate a shared array `probe_array` and transmit a single byte of data.

```

1  uint8_t probe_array[256 * 4096] = {1};
2
3  void cc_setup() {
4      for (int i = 0; i < 256; i++)
5          _mm_clflush(probe_array + i * 4096);
6      _mm_mfence();
7  }
8
9  void cc_transmit(int value) {
10     if (value < 0)
11         return;
12     volatile uint8_t *ptr = probe_array + value * 4096;
13     uint8_t t = *ptr;
14 }
15
16 int cc_receive() {
17     int junk = 0;
18     _mm_mfence();
19
20     for (int i = 0; i < 256; i++) {
21         volatile uint8_t *ptr = probe_array + i * 4096;
22         uint64_t start = __rdtscp(&junk);
23         uint8_t v = *ptr;
24         uint64_t latency = __rdtscp(&junk) - start;
25
26         if (latency < THRESHOLD)
27             return i;
28     }
29     return -1;
30 }

```

Listing 2.5. Implementation of a cache-based covert channel. For a color version of this listing, see www.iste.co.uk/prouff/cryptography1.zip

`cc_setup()` prepares the array for use by flushing 256 locations in the array from the cache (line 5). To transmit a byte, the transmitter invokes `cc_transmit()` (lines 9–14), which accesses one of the 256 locations flushed in the setup. Finally, to receive the value of the byte, the receiver invokes `cc_receive()` (lines 17–30), which reloads each of the 256 locations, returning the value that matches the first found in the cache.

2.7. Transient-execution attacks

Transient execution is a term for speculatively executing instructions only to squash them later when discovering they are not part of the nominal program execution. Transient execution, which is a natural outcome of out-of-order and speculative execution, does not change the architectural state of the processor, and is transparent to the executing program. However, it does leave footprints in the microarchitectural state of the processor. Transient-execution attacks aim at observing these footprints in order to bypass security checks and recover secret information. Specifically, in a transient execution attack, the adversary aims to cause the processor to transiently bypass a security test and transmit a secret value through a microarchitectural covert channel.

Transient-execution attacks are classified based on the main cause of transient execution which they exploit. Meltdown-type attacks exploit execution past an instruction that causes a trap, whereas Spectre-type attacks exploit transient execution caused by misspeculation. We first describe one variant of the Spectre attack and then proceed to Meltdown-type attacks.

2.7.1. The Spectre attack

The Spectre attack exploits transient-execution caused by branch misprediction to overcome software-enforced security boundaries. Listing 2.6 shows a proof of concept of a Spectre attack. The victim is the function `victim` (lines 1 to 5), which checks if `index` is in bounds, returning the contents from the array if it is, otherwise returning a constant 0. Note that in type-safe languages, such as JavaScript, this test is implicit in any array access.

The attack code is the function `spectre`. The code needs to set up the branch predictor so that the victim branch at line 2 mispredicts. To this aim, the attack code uses the function `victim_wrapper`, which calls `victim` after setting the branch history with a small loop (lines 9 and 10). The attack code calls `victim_wrapper` twice with an `index` value in range (lines 17 and 18), training the branch predictor that, for the history set in `victim_wrapper`, the branch condition is false.

```

1  int victim(uint64_t index) {
2      if (index >= array_len)
3          return -1;
4      return array[index];
5  }
6
7
8  int victim_wrapper(uint64_t index) {
9      for (int i = 0; i < 128; i++)
10         ;
11     return victim(index);
12 }
13
14
15 int spectre(int64_t index) {
16     volatile int junk = 0;
17     junk = victim_wrapper(0);
18     junk = victim_wrapper(0);
19
20     cc_setup();
21
22     _mm_clflush(&array_len);
23     _mm_mfence();
24     int rv = victim_wrapper(index);
25     cc_transmit(rv);
26
27     return cc_receive();
28 }

```

Listing 2.6. *The Spectre attack. For a color version of this listing, see www.iste.co.uk/prouff/cryptography1.zip*

The attack code then sets up a covert channel (line 20) and evicts the array length from the cache (line 22). It then calls the victim function with an index that is out of bounds (line 24). The branch at line 2 of the victim code aims to prevent array access with such an index. However, because the attacker evicted the array length from the cache, determining the branch condition requires a cache miss that takes a long time. During that time, the processor checks the BHB for a prediction. Based on the prior training that the condition is true (lines 17 and 18), the branch predictor mispredicts, and execution proceeds speculatively as if the branch condition were true.

During the resultant speculative execution, the processor first retrieves `array[index]`, which now is an arbitrary address, under the control of the attacker. The victim function returns this value, which is transmitted through the covert channel (line 25).

Eventually, the processor retrieves `array_len` from memory, computes the condition at line 2, and detects that the branch was mispredicted. The processor then

terminates the speculative execution that started from the misprediction, and proceeds to the correct execution path, returning an error from `victim`. However, at this stage, the value that was speculatively retrieved from beyond the array bounds has already been transmitted through the covert channel. The attack code receives this value (line 27), returning the contents of memory that the victim function was not supposed to access.

The proof-of-concept code in this section demonstrates that an attacker can overcome software-based protections and recover secret data. Such protections are common in scenarios where untrusted code is running within a more trusted environment. For example, web browsers often execute JavaScript code that is downloaded from untrusted web sites. To allow safe execution of such code, the JavaScript runtime environment restricts the downloaded code to prevent its access to sensitive user data. The Spectre attack can overcome this protection.

Several variants of the Spectre attack have been suggested. These exploit different speculation mechanisms within the processors. Specifically, training the BTB to mispredict indirect branches can lead to arbitrary code execution, which can leak sensitive data, for example from an operating system kernel. Similarly, mistraining the return stack buffer, which predicts the addresses functions return to, has been exploited for attacks.

2.7.2. Meltdown-type attacks

Meltdown-type attacks exploit transient execution to overcome hardware-based protection mechanism. Recall that out-of-order execution may proceed past instructions that raise traps. In a Meltdown attack, the attacker issues a memory load from an address that is part of the operating system kernel. Typically, kernel addresses are mapped to all processes, but are protected against access from user-space code.

Accessing the kernel space raises a trap. However, until the trap is processed, younger instructions may execute. When the attack was discovered, if the accessed address was in the L1 cache, the processor performed the access transiently, allowing leaks of kernel data. Further research demonstrated that similar attacks can read other data that the processor handled, allowing a large number of attacks.

It should be noted that all of these attacks can be considered as processor bugs. Indeed, more recent versions of the processors include countermeasures for such attacks.

2.8. Summary

When multiple programs run on the same computer, they share the use of the microarchitectural components. When a program executes, it changes the state of the microarchitectural state, which, in turn, changes the performance of other programs. This creates a channel that can unintentionally leak information between programs. Such leaks can be exploited by attacks, such as Prime+Probe and Flush+Reload, to compromise the security of cryptographic implementations. To protect against such attacks, cryptographic implementations follow the constant-time paradigm, where the program avoids branches and memory accesses that depend on secret data.

Microarchitectural channels can be combined with speculative and out-of-order execution for mounting transient-execution attacks. In these attacks, transient instructions access data that the program is not allowed to access and transmit it through a microarchitectural channel. The Spectre attack exploits transient execution due to misspeculation to bypass software-based security boundaries. Similarly, Meltdown-type attacks exploit transient execution beyond traps to bypass hardware-enforced security checks.

2.9. Notes and further references

Ge et al. (2018) survey microarchitectural side-channel attacks and Lou et al. (2021) survey their application to cryptography. Canella et al. (2019b) survey transient-execution attacks. The two first attacks to exploit microarchitectural channels were published in 2002. Tsunoo et al. (2002) demonstrate a timing attack that exploits cache collisions in Misty (Matsui 1997), whereas Page (2002) suggests a theoretical attack on Data Encryption Standards (DES) (National Institute of Standards and Technology 1999) which use a trace of cache hits and misses.

- Section 2.2.1. The term Prime+Probe is due to Osvik et al. (2006), who develop the attack and propose using pointer chasing over a linked list to reduce noise due to out-of-order execution. Concurrently with them, Percival (2005) describes a very similar technique, but instead of following pointers, he uses arithmetic operations to create dependencies between memory accesses. Brasser et al. (2017) propose using performance counters instead of timing for the probe step and apply an attack against a genome indexing algorithm that uses hash tables. Last, Gulmezoglu et al. (2017) use both L1 and LLC attacks to detect applications in the cloud.

- Section 2.2.2. The attack in this section follows the approach of Osvik et al. (2006). Also, see this reference for information on the second round attack. Bonneau (2006) proposes a similar attack on the final round of AES, which can recover the full key in a single round. Similar attacks have been proposed against other table-based ciphers (Zhao and Wang 2010; Nguyen et al. 2012; Genkin et al. 2019; Chuengsatansup et al. 2022).

– Section 2.2.3. The first Prime+Probe attacks on the LLC were published in 2015 (Irazoqui et al. 2015a; Liu et al. 2015). The attack has been demonstrated from browsers (Genkin et al. 2018) and across VMs (Ronen et al. 2018). Algorithm 2.1 is based on Liu et al. (2015). Vila et al. (2019) proposes a linear-time algorithm for finding eviction sets. Due to the importance of finding eviction sets for mounting attacks, several secure cache design aim to prevent this step (Werner et al. 2019; Tan et al. 2020; Saileshwar and Qureshi 2021). However, techniques for finding eviction sets for such caches have also been shown (Purnal et al. 2021b). Intel LLCs consist of multiple subcaches called slices. The processor uses a proprietary hash function to select the slice in which a memory location is cached. While the hash function is not public, several techniques for reverse engineering it have been published (Hund et al. 2013; Yarom et al. 2015; Maurice, et al. 2015a; Inci et al. 2016; McCalpin 2021).

– Section 2.2.4. Prime+Abort (Disselkoeen et al. 2017) uses Intel TSX instead of probing. TSX has also been used for defending against cache attacks (Shih et al. 2017; Gruss et al. 2017). Prime+Scope is due to Purnal et al. (2021b). The first published cache occupancy attacks appeared in 2015 (Maurice et al. 2015b; Oren et al. 2015). The attack has been used for website fingerprinting (Shusterman et al. 2019, 2021). Cook et al. (2022) argue that cache contention is not the sole contributor for cache occupancy attacks and show that TLB shutdowns also contribute to the attack. Because they do not rely on eviction sets, cache occupancy attacks are effective against secure cache and can be used for attacking cryptographic implementations (Genkin et al. 2023).

– Section 2.3. Shared memory and cache flushes are the enabler of the Flush+Reload attack (Yarom and Falkner 2014; Gullasch et al. 2011). The attack has been applied in various contexts, including between processes, containers and VMs (Yarom and Falkner 2014; Zhang et al. 2014; Irazoqui et al. 2014), and against multiple cryptographic schemes (Benger et al. 2014; Irazoqui et al. 2014; Irazoqui et al. 2015b; Groot Bruinderink et al. 2016; Pessl et al. 2017). Zhang et al. (2016) propose an Arm implementation that uses system calls to overcome the absence of user-space flush instructions on Arm.

– Section 2.3.2. The demonstrated attack is based on Yarom and Falkner (2014).

– Section 2.3.3. Gruss et al. (2016) propose the Flush+Flush attack. Several works investigated improving its accuracy (Didier and Maurice 2021; Ding et al. 2022). The Evict+Reload technique is due to Gruss et al. (2015). Guo et al. (2022) propose the Prefetch+Prefetch attack among several techniques that exploit the prefetch instruction on Intel processors.

– Section 2.3.4. Allan et al. (2016) propose using cache flushes for performance degradation. Aldaya and Brumley (2022) observe that the technique is more effective within two hyperthreads of the same core.

– Section 2.4. Microarchitectural side-channel attacks targeting almost every component of the shared architecture have been proposed. We discuss attacks against the instruction cache (Aci mez and Schindler 2008; Aci mez et al. 2010; Zhang et al. 2012) and the branch predictors (Aci mez et al. 2006; Aci mez et al. 2007; Bhattacharya and Mukhopadhyay 2015; Evtvushkin et al. 2016, 2018). However attacks have also been demonstrated against translation lookaside buffers (Gras et al. 2018; Tatar et al. 2022), random number generators (Evtvushkin and Ponomarev 2016), interconnects (Wu et al. 2012; Paccagnella et al. 2021; Wan et al. 2022), prefetchers (Shin et al. 2018; Zhang et al. 2023b), memory ordering (Irazoqui et al. 2016; Moghimi et al. 2018; Islam et al. 2019), and many more (Aldaya et al. 2019; Kim et al. 2021; Puddu et al. 2021; Zhao et al. 2022; Rokicki et al. 2022).

– Section 2.5. The foundations for protecting cryptographic software from microarchitectural attacks were laid out together with the publication of the early attacks (Bernstein 2005; Osvik et al. 2006) and later formalized (Barthe et al. 2014). The NaCl library (Bernstein et al. 2012) is one of the first principled implementations of constant-time programming. Jancar et al. (2022) survey tools for enforcing constant-time programming and their use in practice.

– Section 2.6. The covert channel demonstrated in this paper is designed for use with transient-execution attacks. It is based on code from the Meltdown (Lipp et al. 2018) and Spectre (Kocher et al. 2019) papers. Transient-execution attacks exploiting other covert channels have been demonstrated, including port contention (Bhattacharyya et al. 2019), branch prediction (Chowdhury et al. 2020), and instruction timing (Zhang et al. 2023a). Outside transient-execution attacks, Ristenpart et al. (2009) use a cache-based covert channel to detect co-residency on a cloud server, and Maurice et al. (2017) investigate the bandwidth of covert channels.

– Section 2.7. Spectre (Kocher et al. 2019) and Meltdown (Lipp et al. 2018) are the first transient-execution attacks to be published. They were discovered in 2017 and publicly disclosed in early 2018. The distinction between Spectre-type and Meltdown-type attacks is due to Canella et al. (2019b). Xiong and Szefer (2021) survey transient-execution attacks and defenses.

– Section 2.7.1. The proof-of-concept code is based on the code in Kocher et al. (2019), with adaptations to use the covert channel from section 2.6 and the branch training of R ttger and Janc (2021). The code implements the Spectre V1 variant, also called Spectre-PHT (Canella et al. 2019b). Because the typical demo shows read out of array bounds, the variant is often referred to as “bounds check bypass”. However, the same variant has been used in other scenarios, exploiting type confusion (Kirzner and Morrison 2021). Other variants exploit indirect branches (Kocher et al. 2019), the return stack buffer (Maisuradze and Rossow 2018; Koruyeh et al. 2018), and memory ordering speculation (Kiriansky and Waldspurger 2018).

– Section 2.7.2. The Meltdown attack (Lipp et al. 2018) is the prototype for a large number of Meltdown-type attacks, including Foreshadow (Van Bulck et al. 2018), microarchitectural data sampling (MDS) (van Schaik et al. 2019; Schwarz et al. 2019; Canella et al. 2019a), CrossTalk (Ragab et al. 2021), and others. These differ by the causes of traps and the type of information that leaks. Instead of leaking the data obtained from the trapped instruction, the Load Value Injection attack (Van Bulck et al. 2020) speculatively injects it into the victim execution, resulting in transient execution of arbitrary code within the victim.

2.10. References

- Aci mez, O. and Schindler, W. (2008). A vulnerability in RSA implementations due to instruction cache analysis and its demonstration on OpenSSL. In *CT-RSA 2008*, Malkin, T. (ed.). Springer, Berlin, Heidelberg.
- Aci mez, O., Ko ,  .K., Seifert, J.-P. (2006). On the power of simple branch prediction analysis. Report 2006/351, Cryptology ePrint Archive [Online]. Available at: <https://eprint.iacr.org/2006/351>.
- Aci mez, O., Ko ,  ., Seifert, J.-P. (2007). Predicting secret keys via branch prediction. In *CT-RSA 2007*, Abe, M. (ed.). Springer, Berlin, Heidelberg.
- Aci mez, O., Brumley, B.B., Grabher, P. (2010). New results on instruction cache attacks. In *CHES 2010*, Mangard, S. and Standaert, F.-X. (eds). Springer, Heidelberg.
- Aldaya, A.C. and Brumley, B.B. (2022). HyperDegrade: From GHz to MHz effective CPU frequencies. *USENIX Security 2022* [Online]. Available at: https://www.usenix.org/system/files/sec22summer_aldaya.pdf.
- Aldaya, A.C., Brumley, B.B., ul Hassan, S., Garc a, C.P., Tuveri, N. (2019). Port contention for fun and profit. In *2019 IEEE Symposium on Security and Privacy*, San Francisco.
- Allan, T., Brumley, B.B., Falkner, K.E., van de Pol, J., Yarom, Y. (2016). Amplifying side channels through performance degradation. In *ACSAC*. ACM, New York.
- Barthe, G., Betarte, G., Campo, J.D., Luna, C.D., Pichardie, D. (2014). System-level non-interference for constant-time cryptography. In *ACM CCS 2014*, Ahn, G.-J., Yung, M., Li, N. (eds). ACM Press, New York.
- Benger, N., van de Pol, J., Smart, N.P., Yarom, Y. (2014). “Ooh aah... just a little bit”: A small amount of side channel can go a long way. In *CHES 2014*, Batina, L. and Robshaw, M. (eds). Springer, Berlin, Heidelberg.
- Bernstein, D.J. (2005). Cache-timing attacks on AES. Paper, 2217245 [Online]. Available at: <https://cr.yp.to/papers.html#cachetiming>.
- Bernstein, D.J., Lange, T., Schwabe, P. (2012). The security impact of a new cryptographic library. In *LATINCRYPT 2012*, Hevia, A. and Neven, G. (eds). Springer, Berlin, Heidelberg.
- Bhattacharya, S. and Mukhopadhyay, D. (2015). Who watches the watchmen? Utilizing performance monitors for compromising keys of RSA on intel platforms. In *CHES 2015*, G neysu, T. and Handschuh, H. (eds). Springer, Berlin, Heidelberg.

- Bhattacharyya, A., Sandulescu, A., Neugschwandtner, M., Sorniotti, A., Falsafi, B., Payer, M., Kurmus, A. (2019). SMOtherSpectre: Exploiting speculative execution through port contention. In *ACM CCS 2019*, Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds). ACM Press, New York.
- Bonneau, J. (2006). Robust final-round cache-trace attacks against AES. Report 2006/374, Cryptology ePrint Archive [Online]. Available at: <https://eprint.iacr.org/2006/374>.
- Brasser, F., Müller, U., Dmitrienko, A., Kostianen, K., Capkun, S., Sadeghi, A. (2017). Software grand exposure: SGX cache attacks are practical. In *WOOT*. USENIX Association, Berkeley.
- Canella, C., Genkin, D., Giner, L., Gruss, D., Lipp, M., Minkin, M., Moghimi, D., Piessens, F., Schwarz, M., Sunar, B., Van Bulck, J., Yarom, Y. (2019a). Fallout: Leaking data on meltdown-resistant CPUs. In *ACM CCS 2019*, Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds). ACM Press, New York.
- Canella, C., Van Bulck, J., Schwarz, M., Lipp, M., von Berg, B., Ortner, P., Piessens, F., Evtushkin, D., Gruss, D. (2019b). A systematic evaluation of transient execution attacks and defenses. In *USENIX Security 2019*, Heninger, N. and Traynor, P. (eds). USENIX Association, Berkeley.
- Chowdhury, M.H.I., Liu, H., Yao, F. (2020). BranchSpec: Information leakage attacks exploiting speculative branch instruction executions. In *ICCD*. IEEE, Hartford.
- Chuengsatiansup, C., Genkin, D., Yarom, Y., Zhang, Z. (2022). Side-channeling the Kalyna key expansion. In *CT-RSA 2022*, Galbraith, S.D. (ed.). Springer, Berlin, Heidelberg.
- Cook, J., Drean, J., Behrens, J., Yan, M. (2022). There's always a bigger fish: A clarifying analysis of a machine-learning-assisted side-channel attack. In *ISCA*. ACM Press, New York.
- Didier, G. and Maurice, C. (2021). Calibration done right: Noiseless Flush+Flush attacks. In *DIMVA*, Bilge, L., Cavallaro, L., Pellegrino, G., Neves, N. (eds). Springer, Cham.
- Ding, R., Zhang, Z., Zhang, X., Gongye, C., Fei, Y., Ding, A.A. (2022). A cross-platform cache timing attack framework via deep learning. In *DATE*. IEEE, Antwerp.
- Disselkoe, C., Kohlbrenner, D., Porter, L., Tullsen, D.M. (2017). Prime+abort: A timer-free high-precision L3 cache attack using intel TSX. In *USENIX Security 2017*, Kirda, E. and Ristenpart, T. (eds). USENIX Association, Berkeley.
- Evtushkin, D. and Ponomarev, D.V. (2016). Covert channels through random number generator: Mechanisms, capacity estimation and mitigations. In *ACM CCS 2016*, Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds). ACM Press, New York.
- Evtushkin, D., Ponomarev, D.V., Abu-Ghazaleh, N.B. (2016). Jump over ASLR: attacking branch predictors to bypass ASLR. In *MICRO*. IEEE, Taipei.
- Evtushkin, D., Riley, R., Abu-Ghazaleh, N.B., Ponomarev, D. (2018). BranchScope: A new side-channel attack on directional branch predictor. In *ASPLOS*. ACM Press, New York.
- Ge, Q., Yarom, Y., Cock, D., Heiser, G. (2018). A survey of microarchitectural timing attacks and countermeasures on contemporary hardware. *Journal of Cryptographic Engineering*, 8(1), 1–27.

- Genkin, D., Pachmanov, L., Tromer, E., Yarom, Y. (2018). Drive-by key-extraction cache attacks from portable code. In *ACNS 18*, Preneel, B. and Vercauteren, F. (eds). Springer, Berlin, Heidelberg.
- Genkin, D., Poussier, R., Sim, R.Q., Yarom, Y., Zhao, Y. (2019). Cache vs. key-dependency: Side channeling an implementation of pilsung. *IACR TCHES*, 2020(1), 231–255.
- Genkin, D., Kosasih, W., Liu, F., Trikalinou, A., Unterluggauer, T., Yarom, Y. (2023). CacheFX: A framework for evaluating cache security. In *ASIACCS 23*. ACM Press, New York.
- Gras, B., Razavi, K., Bos, H., Giuffrida, C. (2018). Translation leak-aside buffer: Defeating cache side-channel protections with TLB attacks. In *USENIX Security 2018*, Enck, W. and Felt, A.P. (eds). USENIX Association, Berkeley.
- Groot Bruinderink, L., Hülsing, A., Lange, T., Yarom, Y. (2016). Flush, gauss, and reload – A cache attack on the BLISS lattice-based signature scheme. In *CHES 2016*, Gierlichs, B. and Poschmann, A.Y. (eds). Springer, Berlin, Heidelberg.
- Gruss, D., Spreitzer, R., Mangard, S. (2015). Cache template attacks: Automating attacks on inclusive last-level caches. In *USENIX Security 2015*, Jung, J. and Holz, T. (eds). USENIX Association, Berkeley.
- Gruss, D., Maurice, C., Wagner, K., Mangard, S. (2016). Flush+Flush: A fast and stealthy cache attack. In *DIMVA*, Caballero, J., Zurutuza, U., Rodríguez, R. (eds). Springer, Cham.
- Gruss, D., Lettner, J., Schuster, F., Ohrimenko, O., Haller, I., Costa, M. (2017). Strong and efficient cache side-channel protection using hardware transactional memory. In *USENIX Security 2017*, Kirda, E. and Ristenpart, T. (eds). USENIX Association, Berkeley.
- Gullasch, D., Bangerter, E., Krenn, S. (2011). Cache games – Bringing access-based cache attacks on AES to practice. In *2011 IEEE Symposium on Security and Privacy*. Oakland.
- Gulmezoglu, B., Eisenbarth, T., Sunar, B. (2017). Cache-base application detection in the cloud using machine learning. Report 2017/245, Cryptology ePrint Archive [Online]. Available at: <https://eprint.iacr.org/2017/245>.
- Guo, Y., Zigerelli, A., Zhang, Y., Yang, J. (2022). Adversarial prefetch: New cross-core cache side channel attacks. In *2022 IEEE Symposium on Security and Privacy*. San Francisco.
- Hund, R., Willems, C., Holz, T. (2013). Practical timing side channel attacks against kernel space ASLR. In *NDSS 2013*. IEEE, Berkeley.
- Inci, M.S., Gülmezoglu, B., Irazoqui, G., Eisenbarth, T., Sunar, B. (2016). Cache attacks enable bulk key recovery on the cloud. In *CHES 2016*, Gierlichs, B. and Poschmann, A.Y. (eds). Springer, Berlin, Heidelberg.
- Irazoqui, G., Inci, M.S., Eisenbarth, T., Sunar, B. (2014). Wait a minute! A fast, cross-VM attack on AES. In *RAID*, Stavrou, A., Bos, H., Portokalidis, G. (eds). Springer, Cham.
- Irazoqui, G., Eisenbarth, T., Sunar, B. (2015a). S\$A: A shared cache attack that works across cores and defies VM sandboxing – and its application to AES. In *2015 IEEE Symposium on Security and Privacy*. San Jose.
- Irazoqui, G., Inci, M.S., Eisenbarth, T., Sunar, B. (2015b). Lucky 13 strikes back. In *ASIACCS 15*, Bao, F., Miller, S., Zhou, J., Ahn, G.-J. (eds). ACM Press, New York.
- Irazoqui, G., Eisenbarth, T., Sunar, B. (2016). Cross processor cache attacks. In *ASIACCS 16*, Chen, X., Wang, X., Huang, X. (eds). ACM Press, New York.

- Islam, S., Moghimi, A., Bruhns, I., Krebbel, M., Gülmezoglu, B., Eisenbarth, T., Sunar, B. (2019). SPOILER: Speculative load hazards boost rowhammer and cache attacks. In *USENIX Security 2019*, Heninger, N. and Traynor, P. (eds). USENIX Association, Berkeley.
- Jancar, J., Fourné, M., Braga, D.D.A., Sabt, M., Schwabe, P., Barthe, G., Fouque, P.-A., Acar, Y. (2022). “They’re not that hard to mitigate”: What cryptographic library developers think about timing attacks. In *2022 IEEE Symposium on Security and Privacy*. San Francisco.
- Kim, J., Jang, H., Lee, H., Lee, S., Kim, J. (2021). UC-Check: Characterizing micro-operation caches in x86 processors and implications in security and performance. In *MICRO*. ACM Press, New York.
- Kiriansky, V. and Waldspurger, C.A. (2018). Speculative buffer overflows: Attacks and defenses. *arXiv/1807.03757*, 12.
- Kirzner, O. and Morrison, A. (2021). An analysis of speculative type confusion vulnerabilities in the wild. In *USENIX Security 2021*, Bailey, M. and Greenstadt, R. (eds). USENIX Association, Berkeley.
- Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T. et al. (2019). Spectre attacks: Exploiting speculative execution. In *2019 IEEE Symposium on Security and Privacy*. San Francisco.
- Koruyeh, E.M., Khasawneh, K.N., Song, C., Abu-Ghazaleh, N.B. (2018). Spectre returns! Speculation attacks using the return stack buffer. In *WOOT*. USENIX Association, Berkeley.
- Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Fogh, A., Horn, J., Mangard, S., Kocher, P., Genkin, D. et al. (2018). Meltdown: Reading kernel memory from user space. In *USENIX Security 2018*, Enck, W. and Felt, A.P. (eds). USENIX Association, Berkeley.
- Liu, F., Yarom, Y., Ge, Q., Heiser, G., Lee, R.B. (2015). Last-level cache side-channel attacks are practical. In *2015 IEEE Symposium on Security and Privacy*. San Jose.
- Lou, X., Zhang, T., Jiang, J., Zhang, Y. (2021). A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography. *Computing Surveys*, 54(6), 1–37.
- Maisuradze, G. and Rossow, C. (2018). ret2spec: Speculative execution using return stack buffers. In *ACM CCS 2018*, Lie, D., Mannan, M., Backes, M., Wang, X. (eds). ACM Press, New York.
- Matsui, M. (1997). New block encryption algorithm MISTY. In *FSE’97*, Biham, E. (ed.). Springer, Berlin, Heidelberg.
- Maurice, C., Le Scouarnec, N., Neumann, C., Heen, O., Francillon, A. (2015a). Reverse engineering Intel last-level cache complex addressing using performance counters. In *Research in Attacks, Intrusions, and Defenses. RAID 2015*, Bos, H., Monrose, F., Blanc, G. (eds). Springer, Berlin, Heidelberg.
- Maurice, C., Neumann, C., Heen, O., Francillon, A. (2015b). C5: Cross-cores cache covert channel. In *DIMVA*, Almgren, M., Gulisano, V., Maggi, F. (eds). Springer, Cham.
- Maurice, C., Weber, M., Schwarz, M., Giner, L., Gruss, D., Boano, C.A., Mangard, S., Römer, K. (2017). Hello from the other side: SSH over robust cache covert channels in the cloud. In *NDSS 2017*. The Internet Society, San Diego.

- McCalpin, J. (2021). Mapping addresses to L3/CHA slices in Intel processors. ACELab Technical Report, University of Texas at Austin, Austin [Online]. Available at: <https://repositories.lib.utexas.edu/handle/2152/86210>.
- Moghimi, A., Eisenbarth, T., Sunar, B. (2018). MemJam: A false dependency attack against constant-time crypto implementations in SGX. In *CT-RSA 2018*, Smart, N.P. (ed.). Springer, Berlin, Heidelberg.
- National Institute of Standards and Technology (1999). FIPS PUB 46-3 data encryption standard (DES). Standard, NIST, Gaithersburg [Online]. Available at: <https://csrc.nist.gov/files/pubs/fips/46-3/final/docs/fips46-3.pdf>.
- Nguyen, P.H., Rebeiro, C., Mukhopadhyay, D., Wang, H. (2012). Improved differential cache attacks on SMS4. In *Inscrypt*, Kutyłowski, M. and Yung, M. (eds). Springer, Berlin, Heidelberg.
- Oren, Y., Kemerlis, V.P., Sethumadhavan, S., Keromytis, A.D. (2015). The spy in the sandbox: Practical cache attacks in JavaScript and their implications. In *ACM CCS 2015*, Ray, I., Li, N., Kruegel, C. (eds). ACM Press, New York.
- Osvik, D.A., Shamir, A., Tromer, E. (2006). Cache attacks and countermeasures: The case of AES. In *CT-RSA 2006*, Pointcheval, D. (ed.). Springer, Berlin, Heidelberg.
- Paccagnella, R., Luo, L., Fletcher, C.W. (2021). Lord of the ring(s): Side channel attacks on the CPU on-chip ring interconnect are practical. In *USENIX Security 2021*, Bailey, M. and Greenstadt, R. (eds). USENIX Association, Berkeley.
- Page, D. (2002). Theoretical use of cache memory as a cryptanalytic side-channel. Report 2002/169, Cryptology ePrint Archive [Online]. Available at: <https://eprint.iacr.org/2002/169>.
- Percival, C. (2005). Cache missing for fun and profit. In *BSDCan 2005*, Ottawa, CA [Online]. Available at: <https://www.daemonology.net/papers/htt.pdf>.
- Pessl, P., Bruinderink, L.G., Yarom, Y. (2017). To BLISS-B or not to be: Attacking strong swan's implementation of post-quantum signatures. In *ACM CCS 2017*, Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds). ACM Press, New York.
- Puddu, I., Schneider, M., Haller, M., Capkun, S. (2021). Frontal attack: Leaking control-flow in SGX via the CPU frontend. In *USENIX Security 2021*, Bailey, M. and Greenstadt, R. (eds). USENIX Association, Berkeley.
- Purnal, A., Giner, L., Gruss, D., Verbaauwhede, I. (2021a). Systematic analysis of randomization-based protected cache architectures. In *2021 IEEE Symposium on Security and Privacy*. San Francisco.
- Purnal, A., Turan, F., Verbaauwhede, I. (2021b). Prime+scope: Overcoming the observer effect for high-precision cache contention attacks. In *ACM CCS 2021*, Vigna, G. and Shi, E. (eds). ACM Press, New York.
- Ragab, H., Milburn, A., Razavi, K., Bos, H., Giuffrida, C. (2021). CrossTalk: Speculative data leaks across cores are real. In *2021 IEEE Symposium on Security and Privacy*. San Francisco.
- Ristenpart, T., Tromer, E., Shacham, H., Savage, S. (2009). Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds. In *ACM CCS 2009*, Al-Shaer, E., Jha, S., Keromytis, A.D. (eds). ACM Press, New York.

- Rokicki, T., Maurice, C., Botvinnik, M., Oren, Y. (2022). Port contention goes portable: Port contention side channels in web browsers. In *ASIACCS 22*, Suga, Y., Sakurai, K., Ding, X., Sako, K. (eds). ACM Press, New York.
- Ronen, E., Paterson, K.G., Shamir, A. (2018). Pseudo constant time implementations of TLS are only pseudo secure. In *ACM CCS 2018*, Lie, D., Mannan, M., Backes, M., Wang, X. (eds). ACM Press, New York.
- Röttger, S. and Janc, A. (2021). A Spectre proof-of-concept for a Spectre-proof web [Online]. Available at: <https://security.googleblog.com/2021/03/a-spectre-proof-of-concept-for-spectre.html>.
- Saileshwar, G. and Qureshi, M.K. (2021). MIRAGE: Mitigating conflict-based cache attacks with a practical fully-associative design. In *USENIX Security 2021*, Bailey, M. and Greenstadt, R. (eds). USENIX Association, Berkeley.
- van Schaik, S., Milburn, A., Österlund, S., Frigo, P., Maisuradze, G., Razavi, K., Bos, H., Giuffrida, C. (2019). RIDL: Rogue in-flight data load. In *2019 IEEE Symposium on Security and Privacy*. San Francisco.
- Schwarz, M., Lipp, M., Moghimi, D., Van Bulck, J., Stecklina, J., Prescher, T., Gruss, D. (2019). ZombieLoad: Cross-privilege-boundary data sampling. In *ACM CCS 2019*, Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds). ACM Press, New York.
- Shih, M.-W., Lee, S., Kim, T., Peinado, M. (2017). T-SGX: Eradicating controlled-channel attacks against enclave programs. In *NDSS 2017*. The Internet Society, San Diego.
- Shin, Y., Kim, H. C., Kwon, D., Jeong, J.-H., Hur, J. (2018). Unveiling hardware-based data prefetcher, a hidden source of information leakage. In *ACM CCS 2018*, Lie, D., Mannan, M., Backes, M., Wang, X. (eds). ACM Press, New York.
- Shusterman, A., Kang, L., Haskal, Y., Meltser, Y., Mittal, P., Oren, Y., Yarom, Y. (2019). Robust website fingerprinting through the cache occupancy channel. In *USENIX Security 2019*, Heninger, N. and Traynor, P. (eds). USENIX Association, Berkeley.
- Shusterman, A., Agarwal, A., O'Connell, S., Genkin, D., Oren, Y., Yarom, Y. (2021). Prime+probe 1, JavaScript 0: Overcoming browser-based side-channel defenses. In *USENIX Security 2021*, Bailey, M. and Greenstadt, R. (eds). USENIX Association, Berkeley.
- Tan, Q., Zeng, Z., Bu, K., Ren, K. (2020). PhantomCache: Obfuscating cache conflicts with localized randomization. In *NDSS 2020*. The Internet Society, San Diego.
- Tatar, A., Trujillo, D., Giuffrida, C., Bos, H. (2022). TLB;DR: Enhancing TLB-based attacks with TLB desynchronized reverse engineering. In *USENIX Security 2022*, Butler, K.R.B. and Thomas, K. (eds). USENIX Association, Berkeley.
- Tsunoo, Y., Tsujihara, E., Minematsu, K., Miyauchi, H. (2002). Cryptanalysis of block ciphers implemented on computers with cache. In *ISITA*, Walter, C.D., Koç, Ç.K., Paar, C. (eds). Springer, Berlin, Heidelberg.
- Van Bulck, J., Minkin, M., Weisse, O., Genkin, D., Kasikci, B., Piessens, F., Silberstein, M., Wenisch, T.F., Yarom, Y., Strackx, R. (2018). Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution. In *USENIX Security 2018*, Enck, W. and Felt, A.P. (eds). USENIX Association, Berkeley.

- Van Bulck, J., Moghimi, D., Schwarz, M., Lipp, M., Minkin, M., Genkin, D., Yarom, Y., Sunar, B., Gruss, D., Piessens, F. (2020). LVI: Hijacking transient execution through microarchitectural load value injection. In *2020 IEEE Symposium on Security and Privacy*. San Francisco.
- Vila, P., Köpf, B., Morales, J.F. (2019). Theory and practice of finding eviction sets. In *2019 IEEE Symposium on Security and Privacy*. San Francisco.
- Wan, J., Bi, Y., Zhou, Z., Li, Z. (2022). MeshUp: Stateless cache side-channel attack on CPU mesh. In *2022 IEEE Symposium on Security and Privacy*. San Francisco.
- Werner, M., Unterluggauer, T., Giner, L., Schwarz, M., Gruss, D., Mangard, S. (2019). ScatterCache: Thwarting cache attacks via cache set randomization. In *USENIX Security 2019*, Heninger, N. and Traynor, P. (eds). USENIX Association, Berkeley.
- Wu, Z., Xu, Z., Wang, H. (2012). Whispers in the hyper-space: High-speed covert channel attacks in the cloud. In *USENIX Security 2012*, Kohno, T. (ed.). USENIX Association, Berkeley.
- Xiong, W., Szefer, J. (2021). Survey of transient execution attacks and their mitigations. *Computing Surveys*, 54(3), 1–36.
- Yarom, Y. and Falkner, K. (2014). FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In *USENIX Security 2014*, Fu, K. and Jung, J. (eds). USENIX Association, Berkeley.
- Yarom, Y., Ge, Q., Liu, F., Lee, R.B., Heiser, G. (2015). Mapping the intel last-level cache. Report 2015/905, Cryptology ePrint Archive [Online]. Available at: <https://eprint.iacr.org/2015/905>.
- Zhang, Y., Juels, A., Reiter, M.K., Ristenpart, T. (2012). Cross-VM side channels and their use to extract private keys. In *ACM CCS 2012*, Yu, T., Danezis, G., Gligor, V.D. (eds). ACM Press, New York.
- Zhang, Y., Juels, A., Reiter, M.K., Ristenpart, T. (2014). Cross-tenant side-channel attacks in PaaS clouds. In *ACM CCS 2014*, Ahn, G.-J., Yung, M., Li, N. (eds). ACM Press, New York.
- Zhang, X., Xiao, Y., Zhang, Y. (2016). Return-oriented flush-reload side channels on ARM and their implications for android devices. In *ACM CCS 2016*, Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds). ACM Press, New York.
- Zhang, Z., Barthe, G., Chuengsatiansup, C., Schwabe, P., Yarom, Y. (2023a). Ultimate SLH: Taking speculative load hardening to the next level. In *USENIX Security 2023*. USENIX Association, Berkeley.
- Zhang, Z., Tao, M., O'Connell, S., Chuengsatiansup, C., Genkin, D., Yarom, Y. (2023b). BunnyHop: Exploiting the instruction prefetcher. In *USENIX Security 2023*. USENIX Association, Berkeley.
- Zhao, X. and Wang, T. (2010). Improved cache trace attack on AES and CLEFIA by considering Cache miss and S-box misalignment. Report 2010/056, Cryptology ePrint Archive [Online]. Available at: <https://eprint.iacr.org/2010/056>.
- Zhao, Z.N., Morrison, A., Fletcher, C.W., Torrellas, J. (2022). Binoculars: Contention-based side-channel attacks exploiting the page walker. In *USENIX Security 2022*, Butler, K.R.B. and Thomas, K. (eds). USENIX Association, Berkeley.

